Python Introduction IMCBio

Valentine Gilbart

2026-04-29

Table of contents

Summary and setup 3
- Summary . . . 3
  - The Scenario . . . 3
  - Data Format . . . 3
  - Aims . . . 4
- Install Python . . . 4
- Obtain lesson materials . . . 5

1 Lesson 1 - Introduction, representing, manipulating and visualising data 6

2 Introduction 7
- 2.1 Aim of the class . . . 7
- 2.2 What is Python? . . . 7
- 2.3 Why use Python? . . . 8
- 2.4 How can I program in Python? . . . 8
  - 2.4.1 Interactive mode . . . 8
  - 2.4.2 Script mode . . . 9

3 Basic concepts 11
- 3.1 Values and variables . . . 11
- 3.2 Function calls . . . 14
- 3.3 Getting help . . . 17
- 3.4 Comment your code . . . 19

4 Exploring the dataset 21
- 4.1 Importing a python package . . . 21
- 4.2 Loading the patient file . . . 22
- 4.3 Notes on types, attributes and methods . . . 25
- 4.4 Describing the dataset . . . 27
- 4.5 Slicing data . . . 32
- 4.6 Slice with labels . . . 37
- 4.7 Filter based on conditions . . . 39
- 4.8 Analysing data . . . 45
- 4.9 Exercise . . . 55

5 Visualizing data **56**
5.1 Matplotlib package . . . 56
5.2 Function or object-oriented . . . 57
5.3 Matplotlib anatomy . . . 59
5.4 Grouping plots . . . 64
5.5 Save a figure . . . 65
5.6 Matplotlib documentation . . . 66
5.7 Exercise . . . 66

6 Storing multiple values in lists **67**
6.1 Creating a list . . . 67
6.2 Lists are mutable . . . 69
6.3 Nested lists . . . 71
6.4 Some list methods . . . 72
6.5 Exercise . . . 74

7 Repeating actions with loops **75**
7.1 How to use loops . . . 75
7.2 Notes on indentation . . . 79
7.3 Loops and updating variables . . . 80
7.4 Notes on iterators . . . 81
7.5 Exercise . . . 84

8 Analyzing data from multiple files **85**
8.1 Exercise . . . 88

9 Conclusion **89**

References **90**

10 Lesson 2 - Conditional, Functions, User-input and Errors **91**

11 Introduction **92**
11.1 Aim of the class . . . 92

12 Making choices **93**
12.1 Conditionals . . . 93
12.2 Comaparison and membership operators . . . 94
12.3 `elif` statements . . . 96
12.4 Logical operators . . . 97
12.5 Checking our data . . . 98
12.6 Exercise . . . 100

13 Storing multiple values in dictionaries **103**

14 Creating functions **105**
14.1 Syntax . . . 106
14.2 Documentation . . . 107
14.3 Arguments . . . 108
14.4 Output . . . 109
14.5 Variable Scope . . . 111
14.6 Application on our data . . . 112
14.7 Exercise . . . 114

15 Command-Line programs **115**
15.1 Creating a python script . . . 115
15.2 User-defined input . . . 117
15.2.1 input . . . 117
15.2.2 sys.argv . . . 118
15.2.3 argparse . . . 119
15.3 Exercise . . . 120
15.4 Handle multiple filenames . . . 120
15.5 Testing the code . . . 122

16 Errors and Exceptions **124**
16.1 Tracebacks . . . 124
16.2 Type of Exceptions . . . 125
16.3 Exercise . . . 126

17 Defensive Programming **130**
17.1 Exceptions Handling . . . 130
17.2 Exercise . . . 132

18 Debugging **134**
18.1 Know what the code is supposed to do . . . 134
18.2 Make it Fail Everytime . . . 135
18.3 Make it Fail Fast . . . 135
18.4 Change One Thing at a Time . . . 136
18.5 Keep Track of What You've Done . . . 136
18.6 Ask for help . . . 136

19 Final tips and resources **138**

References **139**

Summary and setup

Summary

This course is part of the IMCBio PhD Program courses.

The best way to learn how to program is to do something useful, so this introduction to Python is built around a common scientific task: **data analysis**.

The Scenario

We received a CSV spreadsheet about a clinical trial data. The drug tested promises to cure arthritis inflammation flare-ups after only 3 weeks since initially taking the medication!

The CSV file contains the number of inflammation flare-ups per day for the 60 patients in the initial clinical trial, with the trial lasting 40 days. Each row corresponds to a patient, and each column corresponds to a day in the trial. Once a patient has their first inflammation flare-up they take the medication and wait a few weeks for it to take effect and reduce flare-ups.

To see how effective the treatment is we would like to:

1. Calculate the average inflammation per day across all patients.
2. Plot the result to discuss and share with colleagues.

Data Format

The data sets are stored in comma-separated values (CSV) format:

- each row holds information for a single patient,
- columns represent successive days.

The first four rows of our first file look like this:

```
Day1,Day2,Day3,Day4,Day5,Day6,Day7,Day8,Day9,Day10,Day11,Day12,Day13,Day14,Day15,Day16,Day17
Patient1,0,0,1,3,1,2,4,7,8,3,3,3,10,5,7,4,7,7,12,18,6,13,11,11,7,7,4,6,8,8,4,4,5,7,3,4,2,3,0
Patient2,0,1,2,1,2,1,3,2,2,6,10,11,5,9,4,4,7,16,8,6,18,4,12,5,12,7,11,5,11,3,3,5,4,4,5,5,1,1
Patient3,0,1,1,3,3,2,6,2,5,9,5,7,4,5,4,15,5,11,9,10,19,14,12,17,7,12,11,7,4,2,10,5,4,2,2,3,2
```

Each number represents the number of inflammation flare-ups that a particular patient experienced on a given day.

For example, value "6" at row 4 column 8 of the data set above means that the third patient was experiencing inflammation six times on the seventh day of the clinical study.

In order to analyze this data and report to our colleagues, we'll have to learn a little bit about programming.

Aims

At the end of this course, you will: - Be familiar with the Python environment - Understand the major data types in Python - Manipulate variables with operators and built-in functions - Create simple functions - Upload, modify and download files into Python - Install and import packages - Basic use of pandas (manipulate data) and matplotlib.pyplot (visualize data)

Install Python

For this course, you will need your computer and a way to work with Python. I have two possible solutions for you, either:

A) install both Python (v3 or above), and an IDE (an improved text editor) on your computer. So that we are all on the same page, I recommend installing Visual Studio Code. If for some reason, you are already familiar with another IDE that can be used for Python, you can work with it, but I won't be able to help you as much with it.

NB: It can be useful for your future works to make sure that you have managed to correctly install Python and an IDE on your computer.

OR

B) create a GitHub account. Then create a new codespace with the repository vgilbart/python-intro (everything else should be default). With a bit of patience, you should end up with a Visual Studio Code window (looking somewhat like this).

NB: This does not require installing anything on your computer. This is a free solution up to 60 hours and 15 GB of computing per month (we won't do as much during this class!).

Obtain lesson materials

1. Download inflammation-data.zip.
2. Create a folder called `swc-python` on your Desktop.
3. Move downloaded files to `swc-python`.
4. Unzip the files.

You should see the folder called `data` in the `swc-python` directory on your Desktop.

1 Lesson 1 - Introduction, representing, manipulating and visualising data

2 Introduction

2.1 Aim of the class

At the end of this class, you will:

- Be familiar with the Python environment
- Understand some major data types in Python
- Manipulate variables with built-in functions
- Manipulate data from a file
- Visualize data from a file
- Create loops

2.2 What is Python?

Python is a programming language first released in 1991 and implemented by Guido van Rossum.

It is widely used, with various applications, such as:

- software development
- web development
- data analysis
- …

It supports different types of programming paradigms (i.e. way of thinking) including the procedural programming paradigm. In this approach, the program moves through a linear series of instructions.

```
# Create a string seq
seq = 'ATGAAGGGTCC'
# Call the function len() to retrieve the length of the string
size = len(seq)
# Call the function print() to print a text
print('The sequence has', size, 'bases.')
```

```
The sequence has 11 bases.
```

2.3 Why use Python?

- Easy-to-use and easy-to-read syntax
- Large standard library for many applications (`pandas` for tables, `matplotlib` for graphs, `scikit-learn` for machine learning…)
- Interactive mode making it easy to test short snippets of code
- Large community (stackoverflow)

2.4 How can I program in Python?

Python is an interpreted language, this means that it is not directly compiled into machine code (binary instructions that the computer hardware understands). It is executed by an interpreter program that "translates" each line of the code, into instructions that the computer can understand. By extension, the interpreter that is able to read Python scripts is also called Python. So, whenever you want your Python code to run, you must call the Python interpreter.

2.4.1 Interactive mode

One way to launch the Python interpreter is to type the following, on the command line of a terminal:

```
python3
```

> **Note**
>
> You can also try `python`, `/usr/bin/env python3`, `/usr/bin/python3`… There are many ways to call python!
> You can see where your current python is located by running `which python3`.

From this, you can start using python interactively, e.g. run:

```
print("Hello world")
```

```
Hello world
```

To get out of the Python interpreter, type `quit()`or `exit()`, followed by `enter`. Alternatively, on Linux/Mac press `[ctrl + d]`, on Windows press `[ctrl + z]`.

```
(base) MB1-4074-A:~ gilbartv$ python3
Python 3.11.5 (main, Sep 11 2023, 08:31:25) [Clang 14.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello world")
Hello world
>>> quit()
(base) MB1-4074-A:~ gilbartv$
```

Figure 2.1: Interactive mode

2.4.2 Script mode

To run a script, create a folder named `script`, in which a file named `intro.py` contains:

```
#!/usr/bin/env python3
# -*- coding: UTF-8 -*-

print("Hello world")
```

and run

```
./script/intro.py
```

You should get the same output as before, that is:

```
Hello world
```

The shebang `#!` followed by the interpreter `/usr/bin/env python3` can be put at the beginning of the script in order to ommit calling `python3` in command-line. If you don't put it, you will have to run `python3 script/intro.py` instead of simply `./script/intro.py`.

The `-*- coding: UTF-8 -*-` specify the type of encoding to use. UTF-8 is used by default (which means that this line in the script is not necessary). This accepts characters from all languages. Other valid encoding are available, such as ascii (English characters only).

> ⚠ Warning
>
> Some common errors can occur at this step:
>
> - `bash: script/intro.py: No such file or directory` i.e. you are not in the right directory to run the file.

Solution: run `ls */` and make sure you can find `script/: intro.py`, if not go to the correct directory by running `cd <insert directory name here>`, e.g. `cd ~/Desktop/swc-python` (`~` is a shortcut for your home directory)

- `bash: script/intro.py: Permission denied` i.e. you don't have the right to execute your script.

  Solution: run `ls -l script/intro.py` and make sure you have at least `-rwx` (read, write, exectute rights) as the first 4 characters, if not run `chmod 744 script/intro.py` to change your rights.

- `bash: python3: command not found` i.e. you don't the python3 shortcut.

  Solution: start writting `python` in your shell, then press the `Tab` key on your keyboard, it should try to autocomplete and give you a set of `python` shortcuts available. Once you see a shortcut that works e.g. `python3.11` or `python` instead of our `python3`, you can change the shebang in your script to use it, e.g. `#!/usr/bin/env python` instead of `#!/usr/bin/env python3`. You can also check what path to give by running `which python` or `which python3` and then put this path in the shebang, e.g. `#!/usr/bin/python3` instead of `#!/usr/bin/env python3`.

3 Basic concepts

3.1 Values and variables

Any Python interpreter can be used as a calculator:

```
3 + 5 * 4
```

```
23
```

This is great but not very interesting. To do anything useful with data, we need to assign its value to a *variable*. In Python, we can assign a value to a variable, using the equals sign `=`. For example, we can track the weight of a patient who weighs 60 kilograms by assigning the value `65` to a variable `weight_kg`:

```
weight_kg = 65
```

Figure 3.1: A assigning a value to a variable is analogous to assigning a sticky note to a value.

From now on, whenever we use `weight_kg`, Python will substitute the value we assigned to it. In plain language terms, **a variable is a name for a value**.

A variable can have a short name (like x and y) or a more descriptive name (seq, motif, genome_file). Rules for Python variable names:

- must start with a letter or the underscore character and cannot start with a number

- can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- are case-sensitive (seq, Seq and SEQ are three different variables)
- cannot be any of the Python keywords (run `help('keywords')` to find the list of keywords).

This means that, for example:

- `€or$` is not a valid variable name
- `weight0` is a valid variable name, whereas `0weight` is not
- `weight`, `Weight` and `WEIGHT`, are different variables
- `keywords` are different variables

! Exercise

I want to store the weight of patient 2 in a variable. Are the following variables names legal?

- `2_weight_kg`
- `_weight_kg`
- `weight_kg-2`
- `weight_kg 2`

You can try to assign a value to these variable names to be sure of your answer!

Python knows various types of data. Three common ones are:

- `int` ($\mathbb{Z}$), integers
- `float` ($\mathbb{R}$), real numbers
- `str`, strings or more commonly known as characters

Python will assign a type automatically.

In the example above, variable `weight_kg` has an integer value of `60`. If we want to more precisely track the weight of our patient, we can use a floating point value by executing:

```
weight_kg = 60.3
```

To create a string, we add single or double quotes around some text. To identify and track a patient throughout our study, we can assign each person a unique identifier by storing it in a string:

```
patient_id = '001'
```

Once we have data stored with variable names, we can make use of it in calculations. We may want to store our patient's weight in pounds as well as kilograms:

```
weight_kg = 65.0
weight_lb = 2.2 * weight_kg
```

The expression `2.2 * weight_kg` is evaluated to `143.0`, and then this value is assigned to the variable `weight_lb` (i.e. the sticky noteweight_lbis placed on `143.0`). At this point, each variable is "stuck" to completely distinct and unrelated values.

> **Warning**
>
> The value of `weight_lb` is computed, in the moment of assigning the value, from the current value of `weight_kg`. Modifying `weight_kg` later on will not modify the value of `weight_lb` indirectly (i.e. `weight_lb` is not recomputed every time it is called, its value stays the same)
>
> ```
> print(weight_lb)
>
> weight_kg = 100
>
> print(weight_kg, weight_lb)
> ```
>
> ```
> 143.0
> 100 143.0
> ```
>
>
>
>
> Figure 3.2: Changing the value of weight_kg does not impact the value of weight_lb.

> Since `weight_lb` doesn't 'remember' where its value comes from, it is not updated when we change `weight_kg`.

To carry out common tasks with data and variables in Python, the language provides us with several built-in functions. To display information to the screen, we use the `print` function:

```
print(weight_lb)
print(patient_id)
```

```
143.0
001
```

We just used a function (also known as calling a function)… But what is a function exactly?

3.2 Function calls

A function stores a piece of code that performs a certain task, and that gets run when called. It usually takes some data as input (parameters that are required or optional), and usually returns an output (that can be of any type). Some functions are predefined, like `print()` that prints values, or `len()` that calculates the length of a variable. We will also learn how to create our own later on.

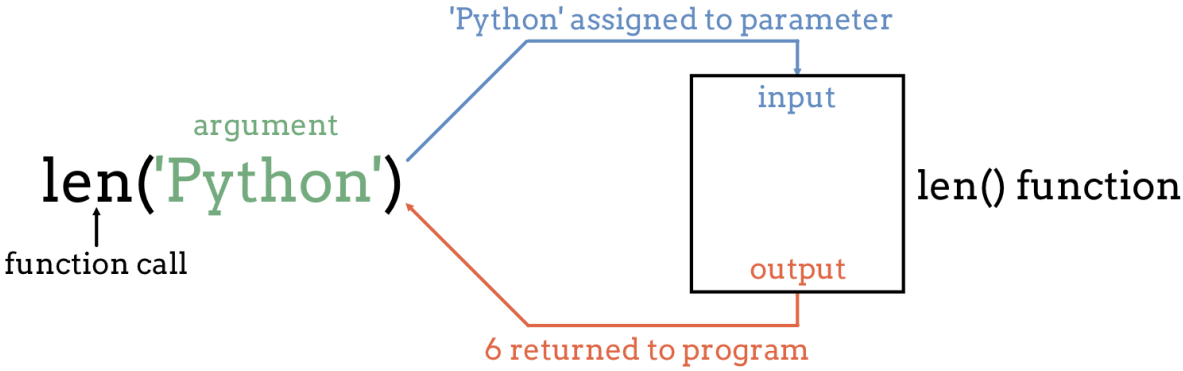


Figure 3.3: Schema of a built-in function from launchcode.org.

To run a function, write its name followed by parentheses. Parameters are added inside the parentheses as follow:

```
print(patient_id)
len(patient_id)
```

```
001
```

```
3
```

We can display multiple things at once using only one `print()` call:

```
print(patient_id, 'weight in kilograms:', weight_kg)
```

```
001 weight in kilograms: 100
```

We can also call a function inside of another function call. For example, Python has a built-in function called `round()` that rounds a value:

```
print(patient_id, 'weight in kilograms:', round(weight_kg))
```

```
001 weight in kilograms: 100
```

Moreover, we can do arithmetic with variables right inside the `print` function:

```
print('weight in pounds:', 2.2 * weight_kg)
```

```
weight in pounds: 220.00000000000003
```

The above command, however, did not change the value of `weight_kg`:

```
print(weight_kg)
```

```
100
```

To change the value of the `weight_kg` variable, we have to **assign** `weight_kg` a new value using the equals `=` sign:

```
weight_kg = 65.0
print('weight in kilograms is now:', weight_kg)
```

```
weight in kilograms is now: 65.0
```

To get more information about a function, use the `help()` function.

Let's see the help for the `round()` function:

```
help(round)
```

```
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.  Otherwise
    the return value has the same type as the number.  ndigits may be negative.
```

Here the function `round()` needs as input `number` a numerical value. As an option, one can add `ndigits` the number of decimal places to be used with digits. If an option is not provided, a default value is given. In the case of the option `ndigits`, `0` is the default. The function returns a numerical value (more specifically a floating point number), that corresponds to the rounded value. This value, just like any other, can be stored in a variable.

```
rounded_weight_kg = round(weight_kg)
print(rounded_weight_kg)
```

```
65
```

> **Note**
>
> If you provide the parameters in the exact same order as they are defined, you don't have to name them. If you name the parameters you can switch their order. As good practice, put all required parameters first.
>
> ```
> round(5.76543, 2)
> ```
>
> ```
> 5.77
> ```

```
round(ndigits = 2, number = 5.76543)
```

```
5.77
```

In Table 3.1 you will find some basic but useful python functions:

Table 3.1: List of useful Python functions.

Function	Description
`print()`	Print into the screen the values given in argument.
`help()`	Execute the built-in help system
`quit()` or `exit()`	Exit from Python
`len()`	Return the length of an object
`round()`	Round a numbers

3.3 Getting help

To get more information about a function or an operator, you can use the `help()` function. For example, in interactive mode, run `help(print)` to display the help of the `print()` function, giving you information about the input and output of this function. If you need information about an operator, you will have to put it into quotes, e.g. `help('+')`

> Browse the help
>
> If the help is long, press `[enter]` to get the next line or `[space]` to get the next 'page' of information.
> To quit the help, press `q`.

Every built-in function has extensive documentation that can also be found online. You can also search the internet when having issues. Paste the last line of your error message or the word "python" and a short description of what you want to do into your favorite search engine and you will usually find several examples where other people have encountered the same problem and came looking for help.

- StackOverflow can be particularly helpful for this: answers to questions are presented as a ranked thread ordered according to how useful other users found them to be
- Ask somebody "in the real world". If you have a colleague or friend with more expertise in Python than you have, show them the problem you are having and ask them for help
- generative AI chatbots

Warning

Copying and pasting code (from a human or a AI chatbot) is risky unless you understand exactly what it is doing!

Warning

You will probably receive some useful guidance by presenting your error message to the chatbot and asking it what went wrong. However, the way this help is provided by the chatbot is different. Answers on StackOverflow have (probably) been given by a human as a direct response to the question asked. But generative AI chatbots, which are based on an advanced statistical model, respond by generating the most likely sequence of text that would follow the prompt they are given.
While responses from generative AI tools can often be helpful, they are not always reliable. These tools sometimes generate plausible but incorrect or misleading information, so (just as with an answer found on the internet) it is essential to verify their accuracy. You need the knowledge and skills to be able to understand these responses, to judge whether or not they are accurate, and to fix any errors in the code it offers you.

Note

In addition to asking for help, programmers can use generative AI tools to generate code from scratch; extend, improve and reorganise existing code; translate code between programming languages; figure out what terms to use in a search of the internet; and more. However, there are drawbacks that you should be aware of:

- The models used by these tools have been "trained" on very large volumes of data, much of it taken from the internet, and the responses they produce reflect that training data, and may recapitulate its inaccuracies or biases.
- The environmental costs (energy and water use) of LLMs are a lot higher than other technologies, both during development (known as training) and when an individual user uses one (also called inference). For more information see the AI Environmental Impact Primer developed by researchers at HuggingFace, an AI hosting platform.
- Concerns also exist about the way the data for this training was obtained, with questions raised about whether the people developing the LLMs had permission to use it.
- Other ethical concerns have also been raised, such as reports that workers were exploited during the training process.

Tip

Remember that for this lesson:

- For most problems you will encounter at this stage, help and answers can be found among the first results returned by searching the internet.
- The fundamental knowledge and skills you will learn in this lesson by writing and fixing your own programs are essential to be able to evaluate the correctness and safety of any code you receive from online help or a generative AI chatbot. If you choose to use these tools in the future, the expertise you gain from learning and practicing these fundamentals on your own will help you use them more effectively.
- As you start out with programming, the mistakes you make will be the kinds that have also been made – and overcome! – by everybody else who learned to program before you. Since these mistakes and the questions you are likely to have at this stage are common, they are also better represented than other, more specialised problems and tasks in the data that was used to train generative AI tools. This means that a generative AI chatbot is more likely to produce accurate responses to questions that novices ask, which could give you a false impression of how reliable they will be when you are ready to do things that are more advanced.

Exercise

Read the help of the `print()` function. Print several variables (e.g `print(weight_kg, weight_lb)`). Using one of the parameters, add a separator in between each value.

3.4 Comment your code

Except for the shebang and coding specifications seen before (e.g. inside a string, defined but quotes `'` or `"`), all things after a hashtag `#` character will be ignored by the interpreter until the end of the line. This is used to add comments in your code.

Comments are used to:

- explain assumptions
- justify decisions in the code
- expose the problem being solved
- inactivate a line to help debug
- …

This line is not evaluated:

```
# print("Hello world")
```

This line is evaluated:

```
print("Hello world")
```

```
Hello world
```

This line is evaluated up until the #:

```
print("Hello world") # This is a comment, it is ignored by the interpreter
```

```
Hello world
```

This line is fully evaluated:

```
print("Hello world, # This is not a comment although there is a hashtag!")
```

```
Hello world, # This is not a comment although there is a hashtag!
```

4 Exploring the dataset

4.1 Importing a python package

Basic built-in functions are useful, but what is even more useful is the possibility to use functions that other people have written, and that are available in Python packages (you might also encounter the terms library or module, we will use them equivalently in this course).

A python package contains a set of function to perform specific tasks.

There are built-in packages that come with Python, and there are also third-party packages that you can install and use. For scientific computing, the most commonly used packages are third-party packages like `numpy` for numerical computing, `pandas` for data manipulation and analysis, matplotlib for data visualization, and `scikit-learn` for machine learning.

A package needs to be **installed** to your computer one time. But **loaded** in your script every time you want to use it.

> ⚠ Warning
>
> Installing a package is done outside of the python interpreter, in command line in a terminal.

You can install a package with `pip`. It should have been automatically installed with your python, to make sure that you have it you can run:

```
# In Linux/MacOS
python -m pip --version
# In Windows
py -m pip --version
```

If it does not work, check out pip documentation

To install a package called pandas, you must run:

```
# In Linux/MacOS
python -m pip install pandas
# In Windows
py -m pip install pandas
```

To get more information about pip, check out the full documentation.

When you wish to use a package in a python script, you'll need to import it, by writing inside of you script:

```
import pandas
```

> **Tip**
>
> Importing a library is like getting a piece of lab equipment out of a storage locker and setting it up on the bench. Libraries provide additional functionality to the basic Python package, much like a new piece of equipment adds functionality to a lab space.

4.2 Loading the patient file

Pandas is a package used to work with data sets, in order to easily clean, manipulate, explore and analyze data. Once we've imported the library, we can ask the library to read our data file for us:

```
# Make sure this is the correct path for you! You are in the directory from where you execute
pandas.read_csv('data/inflammation-01.csv', index_col=0)
```

```
/opt/hostedtoolcache/Python/3.10.20/x64/lib/python3.10/site-packages/IPython/core/formatters
  return method()
```

	Day1	Day2	Day3	Day4	Day5	Day6	Day7	Day8	Day9	Day10	Day11	Day12	D
Patient1	0	0	1	3	1	2	4	7	8	3	3	3	
Patient2	0	1	2	1	2	1	3	2	2	6	10	11	
Patient3	0	1	1	3	3	2	6	2	5	9	5	7	
Patient4	0	0	2	0	4	2	2	1	6	7	10	7	
Patient5	0	1	1	3	3	1	3	5	2	4	4	7	
Patient6	0	0	1	2	2	4	2	1	6	4	7	6	
Patient7	0	0	2	2	4	2	2	5	5	8	6	5	
Patient8	0	0	1	2	3	1	2	3	5	3	7	8	
Patient9	0	0	0	3	1	5	6	5	5	8	2	4	
Patient10	0	1	1	2	1	3	5	3	5	8	6	8	
Patient11	0	1	0	0	4	3	3	5	5	4	5	8	
Patient12	0	1	0	0	3	4	2	7	8	5	2	8	
Patient13	0	0	2	1	4	3	6	4	6	7	9	9	
Patient14	0	0	0	0	1	3	1	6	6	5	5	6	
Patient15	0	1	2	1	1	1	4	1	5	2	3	3	
Patient16	0	1	1	0	1	2	4	3	6	4	7	5	
Patient17	0	0	0	0	2	3	6	5	7	4	3	2	
Patient18	0	0	0	1	2	1	4	3	6	7	4	2	
Patient19	0	0	2	1	2	5	4	2	7	8	4	7	
Patient20	0	1	2	0	1	4	3	2	2	7	3	3	
Patient21	0	1	1	3	1	4	4	1	8	2	2	3	
Patient22	0	0	2	3	2	3	2	6	3	8	7	4	
Patient23	0	0	0	3	4	5	1	7	7	8	2	5	
Patient24	0	1	1	1	1	3	3	2	6	3	9	7	
Patient25	0	1	1	1	2	3	5	3	6	3	7	10	
Patient26	0	0	2	1	3	3	2	7	4	4	3	8	
Patient27	0	0	1	2	4	2	2	3	5	7	10	5	
Patient28	0	0	1	1	1	5	1	5	2	2	4	10	
Patient29	0	0	2	2	3	4	6	3	7	6	4	5	
Patient30	0	0	0	1	4	4	6	3	8	6	4	10	
Patient31	0	1	1	0	3	2	4	6	8	6	2	3	
Patient32	0	0	2	3	3	4	5	3	6	7	10	5	
Patient33	0	1	2	2	2	3	6	6	6	7	6	3	
Patient34	0	0	2	1	3	5	6	7	5	8	9	3	
Patient35	0	0	1	2	4	1	5	5	2	3	4	8	
Patient36	0	0	0	3	1	3	6	4	3	4	8	3	
Patient37	0	1	2	2	2	5	5	1	4	6	3	6	
Patient38	0	1	1	2	3	1	5	1	2	2	5	7	
Patient39	0	1	0	3	2	4	1	1	5	9	10	7	
Patient40	0	1	1	3	1	1	5	5	3	7	2	2	
Patient41	0	0	0	2	2	1	3	4	5	5	6	5	
Patient42	0	0	1	3	3	1	2	1	8	9	2	8	
Patient43	0	1	1	3	4	5	2	1	3	7	9	6	
Patient44	0	0	1	3	1	4	3	6	7	8	5	7	
Patient45	0	1	1	3	3	4	4	6	3	4	9	9	
Patient46	0	1	2	2	4	3	1	4	8	9	5	10	
Patient47	0	0	2	3	4	5	4	6	2	9	7	4	
Patient48	0	1	1	3	1	4	6	2	8	2	10	3	
Patient49	0	0	1	3	2	5	1	2	7	6	6	3	
Patient50	0	0	1	2	3	4	5	7	5	4	10	5	
Patient51	0	1	2	1	1	3	5	3	6	3	10	10	
Patient52	0	1	2	2	3	5	2	4	5	6	8	3	

The expression `pandas.read_csv()` is a function call that asks Python to run the function `read_csv()` which belongs to the pandas library.

> **Note**
>
> The dot notation in Python is used most of all as an object attribute/property specifier or for invoking its function. `object.property` will give you the value of `property` from `object`, while `object.function()` will invoke on object function.

We used `pandas.read_csv()` with two parameter:

- the name of the file we want to read. This parameter needs to be a character string, so we put it in quotes.
- `index_col`, an optional parameter that specifies the column number to use as the row names.

Since we haven't told it to do anything else with the function's output, it displays it. To save the data in memory, we need to assign it to a variable:

```
data = pandas.read_csv('data/inflammation-01.csv', index_col=0)
```

This statement doesn't produce any output because we've assigned the output to the variable data. If we want to check that the data have been loaded, we can print the variable's value (to not print everything, we will use `.head()` method):

```
print(data.head(5))
```

```
          Day1  Day2  Day3  Day4  Day5  Day6  Day7  Day8  Day9  Day10  ...  \
Patient1     0     0     1     3     1     2     4     7     8      3  ...
Patient2     0     1     2     1     2     1     3     2     2      6  ...
Patient3     0     1     1     3     3     2     6     2     5      9  ...
Patient4     0     0     2     0     4     2     2     1     6      7  ...
Patient5     0     1     1     3     3     1     3     5     2      4  ...

          Day31  Day32  Day33  Day34  Day35  Day36  Day37  Day38  Day39  Day40
Patient1      4      4      5      7      3      4      2      3      0      0
Patient2      3      5      4      4      5      5      1      1      0      1
Patient3     10      5      4      2      2      3      2      2      1      1
Patient4      3      5      6      3      3      4      2      3      2      1
Patient5      9      6      3      2      2      4      2      0      1      1

[5 rows x 40 columns]
```

> **Note**
>
> A method is a function that is associated with an object. It is called on the object and can access and modify the object's data. In Python, methods are defined within classes and are accessed using dot notation.
> For example, if you have a pandas object called `data`, you can call the `head()` method on it to display the first few rows of the data by writing `data.head()`. The `head()` method is a built-in method of pandas DataFrame objects that returns the first n rows of the DataFrame, where n is specified as an argument (default is 5).
> To access the help of such a method, you can use `help(pandas.DataFrame.head)`, where pandas is the library, DataFrame is the class of the object, and head is the method you want to learn about.
> Methods are a fundamental part of another programming paradigm in Python called object-oriented programming, and allow you to perform operations on objects in a convenient and organized way.

4.3 Notes on types, attributes and methods

In Python, data can be of different types, and the type of data determines what operations can be performed on it. For example, the + operator on two numbers will add them mathematically, but on two strings it will concatenate them.

There is a function in Python to determine the type of a value or variable, it is called `type()`.

```
print(type(65))
print(type(65.0))
print(type("Hello world"))

print(type(weight_kg))

print(type(round))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'float'>
<class 'builtin_function_or_method'>
```

Let's see what is the type of the data we loaded with pandas:

```
print(type(data))
```

```
<class 'pandas.core.frame.DataFrame'>
```

It is called a DataFrame. A DataFrame is a two-dimensional data structure with labeled axes. It is one of the most commonly used data structures in the pandas library for data manipulation and analysis. It can be thought of as what we generally call a table with rows and columns. Each column can contain different types of data (e.g., integers, floats, strings) and each row represents a record or an observation.

It is actually composed of a pandas data type called Series. A Series is a one-dimensional array-like object that can hold any data type (integers, floats, strings, etc.). It is similar to a column in a table. Each element in a Series has an associated index, which allows for easy access and manipulation of the data. A DataFrame is essentially a collection of Series objects that share the same index.

We can access one column of the data with `data.iloc[0]` (the first column has index 0, the second column has index 1, etc.) and check its type:

```
print(data.iloc[0].head(5))
print(type(data.iloc[0]))
```

```
Day1    0
Day2    0
Day3    1
Day4    3
Day5    1
Name: Patient1, dtype: int64
<class 'pandas.core.series.Series'>
```

Notice that the output of `print(data.iloc[0].head(5))` also gives two information: `Name:0, dtype: int64`. `Name:0` indicates the name of the Series, which is 0 in this case (since we didn't specify column names when loading the data, pandas automatically assigns integer names to the columns starting from 0). `dtype: int64` indicates the data type of the values in the Series, which is int64, meaning that the values are 64-bit integers.

We could get these same information by running:

```
print(data.iloc[0].name)
print(type(data.iloc[0].dtype))
```

```
Patient1
<class 'numpy.dtype[int64]'>
```

There are also the index (the rownames) on the left of the output, that are not part of the data but are generated by pandas to help us access the data. The index could be also accessed like so:

```
print(data.iloc[0].index)
print(type(data.iloc[0].index))
```

```
Index(['Day1', 'Day2', 'Day3', 'Day4', 'Day5', 'Day6', 'Day7', 'Day8', 'Day9',
       'Day10', 'Day11', 'Day12', 'Day13', 'Day14', 'Day15', 'Day16', 'Day17',
       'Day18', 'Day19', 'Day20', 'Day21', 'Day22', 'Day23', 'Day24', 'Day25',
       'Day26', 'Day27', 'Day28', 'Day29', 'Day30', 'Day31', 'Day32', 'Day33',
       'Day34', 'Day35', 'Day36', 'Day37', 'Day38', 'Day39', 'Day40'],
      dtype='object')
<class 'pandas.core.indexes.base.Index'>
```

> **Note**
>
> Notice that .index and .dtype are attributes of the Series, not methods, since they are not called with parentheses (). An attribute is a value associated with an object, while a method is a function that is associated with an object and can be called to perform an action on that object. In this case, .index and .dtype are attributes that provide information about the Series, while methods like .head() perform actions on the Series.

4.4 Describing the dataset

There are many useful methods and attributes in pandas that helps describe a DataFrame.

For example, to get the number of rows and the number of colmuns of the DataFrame:

```
print(data.shape) # rows x columns i.e. patients x days
```

```
(60, 40)
```

You get access to the index and column names with:

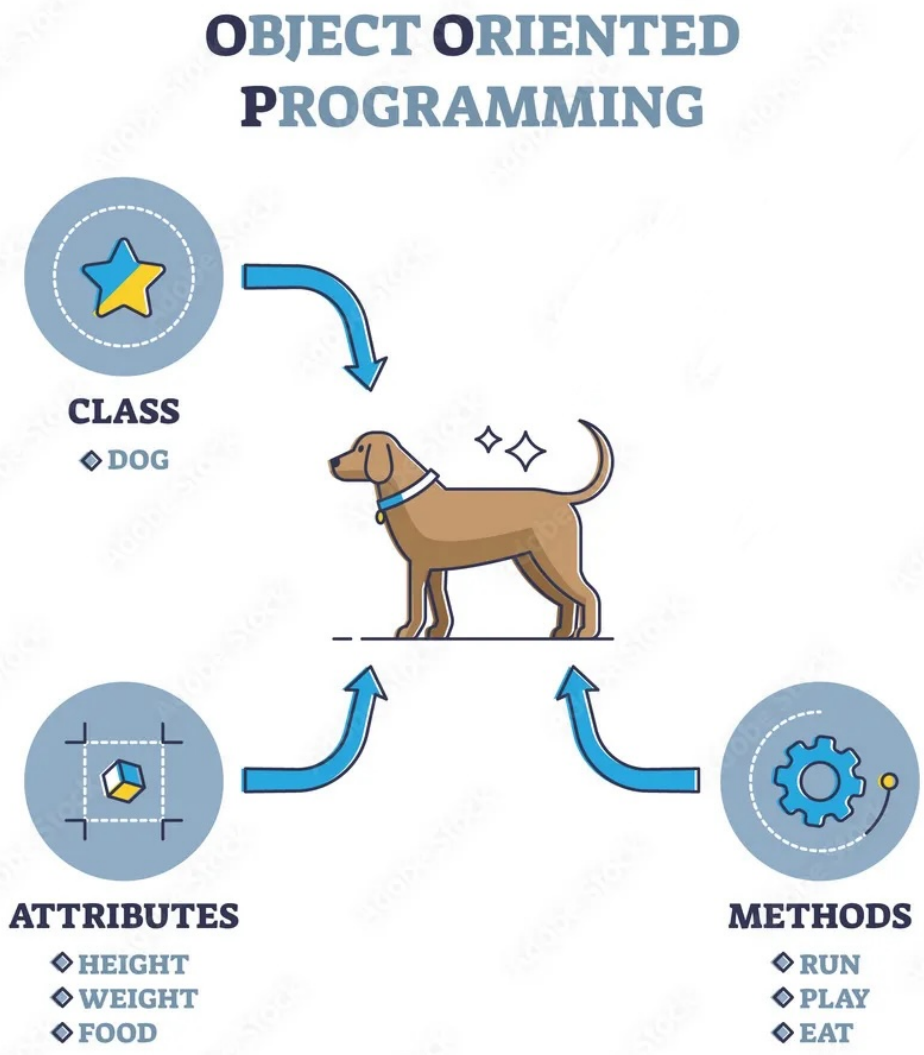


Figure 4.1: Attribute and method schema from medium.com

```python
print(data.columns) # days
print(data.index) # patients
```

```
Index(['Day1', 'Day2', 'Day3', 'Day4', 'Day5', 'Day6', 'Day7', 'Day8', 'Day9',
       'Day10', 'Day11', 'Day12', 'Day13', 'Day14', 'Day15', 'Day16', 'Day17',
       'Day18', 'Day19', 'Day20', 'Day21', 'Day22', 'Day23', 'Day24', 'Day25',
       'Day26', 'Day27', 'Day28', 'Day29', 'Day30', 'Day31', 'Day32', 'Day33',
       'Day34', 'Day35', 'Day36', 'Day37', 'Day38', 'Day39', 'Day40'],
      dtype='object')
Index(['Patient1', 'Patient2', 'Patient3', 'Patient4', 'Patient5', 'Patient6',
       'Patient7', 'Patient8', 'Patient9', 'Patient10', 'Patient11',
       'Patient12', 'Patient13', 'Patient14', 'Patient15', 'Patient16',
       'Patient17', 'Patient18', 'Patient19', 'Patient20', 'Patient21',
       'Patient22', 'Patient23', 'Patient24', 'Patient25', 'Patient26',
       'Patient27', 'Patient28', 'Patient29', 'Patient30', 'Patient31',
       'Patient32', 'Patient33', 'Patient34', 'Patient35', 'Patient36',
       'Patient37', 'Patient38', 'Patient39', 'Patient40', 'Patient41',
       'Patient42', 'Patient43', 'Patient44', 'Patient45', 'Patient46',
       'Patient47', 'Patient48', 'Patient49', 'Patient50', 'Patient51',
       'Patient52', 'Patient53', 'Patient54', 'Patient55', 'Patient56',
       'Patient57', 'Patient58', 'Patient59', 'Patient60'],
      dtype='object')
```

To explore the data set, use the following methods:

```python
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 60 entries, Patient1 to Patient60
Data columns (total 40 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Day1    60 non-null     int64
 1   Day2    60 non-null     int64
 2   Day3    60 non-null     int64
 3   Day4    60 non-null     int64
 4   Day5    60 non-null     int64
 5   Day6    60 non-null     int64
 6   Day7    60 non-null     int64
 7   Day8    60 non-null     int64
 8   Day9    60 non-null     int64
```

```
 9   Day10   60 non-null     int64
 10  Day11   60 non-null     int64
 11  Day12   60 non-null     int64
 12  Day13   60 non-null     int64
 13  Day14   60 non-null     int64
 14  Day15   60 non-null     int64
 15  Day16   60 non-null     int64
 16  Day17   60 non-null     int64
 17  Day18   60 non-null     int64
 18  Day19   60 non-null     int64
 19  Day20   60 non-null     int64
 20  Day21   60 non-null     int64
 21  Day22   60 non-null     int64
 22  Day23   60 non-null     int64
 23  Day24   60 non-null     int64
 24  Day25   60 non-null     int64
 25  Day26   60 non-null     int64
 26  Day27   60 non-null     int64
 27  Day28   60 non-null     int64
 28  Day29   60 non-null     int64
 29  Day30   60 non-null     int64
 30  Day31   60 non-null     int64
 31  Day32   60 non-null     int64
 32  Day33   60 non-null     int64
 33  Day34   60 non-null     int64
 34  Day35   60 non-null     int64
 35  Day36   60 non-null     int64
 36  Day37   60 non-null     int64
 37  Day38   60 non-null     int64
 38  Day39   60 non-null     int64
 39  Day40   60 non-null     int64
dtypes: int64(40)
memory usage: 19.2+ KB
None
```

The `.info()` method tells us that our data set has 60 rows (patients) and 40 columns (days), that there are no missing values (`60 non-null`), and that all the values are integers (`int64`).

```
print(data.describe())
```

```
       Day1       Day2       Day3       Day4       Day5       Day6       Day7  \
count  60.0  60.000000  60.000000  60.000000  60.000000  60.000000  60.000000
```

```
mean    0.0   0.450000   1.116667   1.750000   2.433333   3.150000   3.800000
std     0.0   0.501692   0.738566   1.067628   1.140423   1.387902   1.725187
min     0.0   0.000000   0.000000   0.000000   1.000000   1.000000   1.000000
25%     0.0   0.000000   1.000000   1.000000   1.000000   2.000000   2.000000
50%     0.0   0.000000   1.000000   2.000000   2.000000   3.000000   4.000000
75%     0.0   1.000000   2.000000   3.000000   3.000000   4.000000   5.000000
max     0.0   1.000000   2.000000   3.000000   4.000000   5.000000   6.000000

            Day8       Day9      Day10  ...      Day31      Day32      Day33  \
count  60.000000  60.000000  60.000000  ...  60.000000  60.000000  60.000000
mean    3.883333   5.233333   5.516667  ...   6.066667   5.950000   5.116667
std     1.966600   1.942972   2.281032  ...   2.536958   2.126707   1.637398
min     1.000000   2.000000   2.000000  ...   2.000000   2.000000   2.000000
25%     2.000000   4.000000   3.750000  ...   4.000000   4.000000   4.000000
50%     4.000000   5.000000   6.000000  ...   6.000000   6.000000   5.000000
75%     5.250000   7.000000   7.000000  ...   8.000000   8.000000   6.000000
max     7.000000   8.000000   9.000000  ...  10.000000   9.000000   8.000000

           Day34      Day35      Day36      Day37    Day38      Day39  \
count  60.000000  60.000000  60.000000  60.000000  60.0000  60.000000
mean    3.600000   3.300000   3.566667   2.483333   1.5000   1.133333
std     1.796418   1.778401   1.394501   1.127344   1.1717   0.812334
min     1.000000   1.000000   1.000000   1.000000   0.0000   0.000000
25%     2.000000   2.000000   2.000000   2.000000   0.0000   0.000000
50%     4.000000   3.000000   4.000000   2.000000   1.0000   1.000000
75%     5.000000   5.000000   5.000000   4.000000   3.0000   2.000000
max     7.000000   6.000000   5.000000   4.000000   3.0000   2.000000

           Day40
count  60.000000
mean    0.566667
std     0.499717
min     0.000000
25%     0.000000
50%     1.000000
75%     1.000000
max     1.000000

[8 rows x 40 columns]
```

The `.describe()` method gives us some basic statistics about the data column-wise (i.e. day-wise), such as the mean, standard deviation, minimum and maximum values, and the quartiles.

From the mean, we can already see that at the number of inflammation flare-ups increases over time, to then decrease back close to 0.

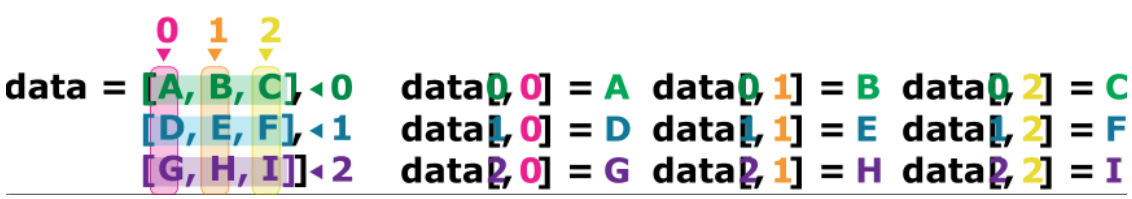
4.5 Slicing data

If we want to get a single number from the DataFrame, we must provide an index in square brackets after the variable name, just as we do in math when referring to an element of a matrix. Our inflammation data has two dimensions, so we will need to use two indices to refer to one specific value:

```
print('first value in data:', data.iloc[0, 0])
print('middle value in data:', data.iloc[29, 19])
```

```
first value in data: 0
middle value in data: 16
```

The expression `data.iloc[29, 19]` accesses the element at row 30, column 20. While this expression may not surprise you, `data.iloc[0, 0]` might.

> **Note**
>
> Programming languages like R start counting at 1 because that's what human beings do. Languages like Python count from 0 because it is closer to the way that computers represent arrays (if you are interested in the historical reasons behind counting indices from zero, you can read Mike Hoye's blog post).
> As a result, if we have an MxN array in Python, its indices go from 0 to M-1 on the first axis and 0 to N-1 on the second. It takes a bit of getting used to, but one way to remember the rule is that the index is how many steps we have to take from the start to get the item we want.
>
> 
>
>
> Figure 4.2: Schema of Python indexes.

`.iloc[29, 19]` selects a single element of an array, but we can select whole sections as well. For example, we can select the first ten days (columns) of values for the first four patients (rows) like this:

```
print(data.iloc[0:4, 0:10])
```

```
          Day1  Day2  Day3  Day4  Day5  Day6  Day7  Day8  Day9  Day10
Patient1     0     0     1     3     1     2     4     7     8      3
Patient2     0     1     2     1     2     1     3     2     2      6
Patient3     0     1     1     3     3     2     6     2     5      9
Patient4     0     0     2     0     4     2     2     1     6      7
```

The slice 0:4 means, "Start at index 0 and go up to, but not including, index 4". Again, the up-to-but-not-including takes a bit of getting used to, but the rule is that the difference between the upper and lower bounds is the number of values in the slice.

We don't have to start slices at 0:

```
print(data.iloc[5:10, 0:10])
```

```
           Day1  Day2  Day3  Day4  Day5  Day6  Day7  Day8  Day9  Day10
Patient6      0     0     1     2     2     4     2     1     6      4
Patient7      0     0     2     2     4     2     2     5     5      8
Patient8      0     0     1     2     3     1     2     3     5      3
Patient9      0     0     0     3     1     5     6     5     5      8
Patient10     0     1     1     2     1     3     5     3     5      8
```

We also don't have to include the upper and lower bound on the slice. If we don't include the lower bound, Python uses 0 by default; if we don't include the upper, the slice runs to the end of the axis, and if we don't include either (i.e., if we use `:` on its own), the slice includes everything:

```
small = data.iloc[:3, 36:]
print('small is:')
print(small)
```

```
small is:
          Day37  Day38  Day39  Day40
Patient1      2      3      0      0
Patient2      1      1      0      1
Patient3      2      2      1      1
```

or even:

```python
data.iloc[:3, :]
data.iloc[:, 36:]
```

```
/opt/hostedtoolcache/Python/3.10.20/x64/lib/python3.10/site-packages/IPython/core/formatters
  return method()
```

	Day37	Day38	Day39	Day40
Patient1	2	3	0	0
Patient2	1	1	0	1
Patient3	2	2	1	1
Patient4	2	3	2	1
Patient5	2	0	1	1
Patient6	4	3	2	1
Patient7	4	1	1	1
Patient8	3	2	2	1
Patient9	2	1	0	0
Patient10	1	2	0	0
Patient11	3	2	2	1
Patient12	4	0	1	1
Patient13	2	3	0	1
Patient14	1	3	0	0
Patient15	2	0	2	0
Patient16	4	2	2	0
Patient17	4	0	2	1
Patient18	3	0	1	0
Patient19	2	0	1	0
Patient20	4	3	1	1
Patient21	4	2	2	0
Patient22	3	0	2	0
Patient23	1	0	1	0
Patient24	4	1	2	0
Patient25	1	0	2	1
Patient26	1	1	1	0
Patient27	3	0	1	1
Patient28	4	3	2	1
Patient29	1	3	1	0
Patient30	2	1	0	1
Patient31	2	3	2	1
Patient32	2	3	0	0
Patient33	2	0	2	1
Patient34	1	2	1	1
Patient35	2	1	1	1
Patient36	4	2	2	0
Patient37	4	3	2	1
Patient38	1	0	1	0
Patient39	1	3	0	1
Patient40	4	3	2	1
Patient41	3	0	0	1
Patient42	2	2	0	0
Patient43	2	1	2	0
Patient44	2	1	0	1
Patient45	4	3	1	1
Patient46	4	1	1	0
Patient47	1	0	0	0
Patient48	2	1	2	1
Patient49	4	3	0	0
Patient50	2	2	2	1
Patient51	4	1	2	0
Patient52	1	3	1	0

It is also possible to use negative index, `-1` will retrieve the last item, `-2` the second last item, etc. For example, to get the value last 10 patient on the 1st day, you could use:

```
print(data.iloc[40:-1, 0])
```

```
Patient41    0
Patient42    0
Patient43    0
Patient44    0
Patient45    0
Patient46    0
Patient47    0
Patient48    0
Patient49    0
Patient50    0
Patient51    0
Patient52    0
Patient53    0
Patient54    0
Patient55    0
Patient56    0
Patient57    0
Patient58    0
Patient59    0
Name: Day1, dtype: int64
```

Tip

One way to remember how slices work is to think of the indices as pointing between characters, with the left edge of the first character numbered 0. Then the right edge of the last character of a string of n characters has index n, for example:

```
  +---+---+---+---+---+---+
  | P | y | t | h | o | n |
  +---+---+---+---+---+---+
  0   1   2   3   4   5   6
 -6  -5  -4  -3  -2  -1
```

The first row of numbers gives the position of the indices 0…6 in the string; the second row gives the corresponding negative indices. The slice from i to j consists of all characters between the edges labeled i and j, respectively.

4.6 Slice with labels

Since we loaded our data with `index_col=0`, the first column of the data is used as row names (i.e. patient names). We can use these names to slice the data instead of using the integer indices. To do this, we need to use `.loc[]` instead of `.iloc[]`:

```
print(data.loc['Patient1', 'Day10'])
print(data.loc['Patient1',])
print(data.loc['Patient1':'Patient10', 'Day10':'Day20'])
print(data.loc[:'Patient10', 'Day20':])
```

```
3
Day1      0
Day2      0
Day3      1
Day4      3
Day5      1
Day6      2
Day7      4
Day8      7
Day9      8
Day10     3
Day11     3
Day12     3
Day13    10
Day14     5
Day15     7
Day16     4
Day17     7
Day18     7
Day19    12
Day20    18
Day21     6
Day22    13
Day23    11
Day24    11
Day25     7
Day26     7
Day27     4
Day28     6
Day29     8
Day30     8
```

```
Day31     4
Day32     4
Day33     5
Day34     7
Day35     3
Day36     4
Day37     2
Day38     3
Day39     0
Day40     0
Name: Patient1, dtype: int64
           Day10  Day11  Day12  Day13  Day14  Day15  Day16  Day17  Day18  \
Patient1       3      3      3     10      5      7      4      7      7
Patient2       6     10     11      5      9      4      4      7     16
Patient3       9      5      7      4      5      4     15      5     11
Patient4       7     10      7      9     13      8      8     15     10
Patient5       4      4      7      6      5      3     10      8     10
Patient6       4      7      6      6      9      9     15      4     16
Patient7       8      6      5     11      9      4     13      5     12
Patient8       3      7      8      8      5     10      9     15     11
Patient9       8      2      4     11     12     10     11      9     10
Patient10      8      6      8     12      5     13      6     13      8

           Day19  Day20
Patient1      12     18
Patient2       8      6
Patient3       9     10
Patient4      10      7
Patient5       6     17
Patient6      18     12
Patient7      10      6
Patient8      18     19
Patient9      17     11
Patient10     16      8
           Day20  Day21  Day22  Day23  Day24  Day25  Day26  Day27  Day28  \
Patient1      18      6     13     11     11      7      7      4      6
Patient2       6     18      4     12      5     12      7     11      5
Patient3      10     19     14     12     17      7     12     11      7
Patient4       7     17      4      4      7      6     15      6      4
Patient5      17      9     14      9      7     13      9     12      6
Patient6      12     12      5     18      9      5      3     10      3
Patient7       6      9     17     15      8      9      3     13      7
Patient8      19     20      8      5     13     15     10      6     10
```

```
Patient9      11      6     16     12      6      8     14      6     13
Patient10      8     18     15     16     14     12      7      3      8

           Day29  ...  Day31  Day32  Day33  Day34  Day35  Day36  Day37  Day38  \
Patient1       8  ...      4      4      5      7      3      4      2      3
Patient2      11  ...      3      5      4      4      5      5      1      1
Patient3       4  ...     10      5      4      2      2      3      2      2
Patient4       9  ...      3      5      6      3      3      4      2      3
Patient5       7  ...      9      6      3      2      2      4      2      0
Patient6      12  ...      8      4      7      3      5      4      4      3
Patient7       8  ...      8      8      4      2      3      5      4      1
Patient8       6  ...      4      9      3      5      2      5      3      2
Patient9      10  ...      4      6      4      7      6      3      2      1
Patient10      9  ...      2      5      4      5      1      4      1      2

           Day39  Day40
Patient1       0      0
Patient2       0      1
Patient3       1      1
Patient4       2      1
Patient5       1      1
Patient6       2      1
Patient7       1      1
Patient8       2      1
Patient9       0      0
Patient10      0      0

[10 rows x 21 columns]
```

It works the same way as `.iloc[]` but with the labels instead of the integer indices. The only difference is that when slicing with labels, the upper bound is included in the slice (i.e. `Patient10` and `Day20` are included in the slices above).

4.7 Filter based on conditions

You can also slice based on conditions. For example, to get the values of the patients on the 4th day that are greater than 2, you can run:

```
data.iloc[:, 3] > 2
```

```
/opt/hostedtoolcache/Python/3.10.20/x64/lib/python3.10/site-packages/IPython/core/formatters
  return method()
```

	Day4
Patient1	True
Patient2	False
Patient3	True
Patient4	False
Patient5	True
Patient6	False
Patient7	False
Patient8	False
Patient9	True
Patient10	False
Patient11	False
Patient12	False
Patient13	False
Patient14	False
Patient15	False
Patient16	False
Patient17	False
Patient18	False
Patient19	False
Patient20	False
Patient21	True
Patient22	True
Patient23	True
Patient24	False
Patient25	False
Patient26	False
Patient27	False
Patient28	False
Patient29	False
Patient30	False
Patient31	False
Patient32	True
Patient33	False
Patient34	False
Patient35	False
Patient36	True
Patient37	False
Patient38	False
Patient39	True
Patient40	True
Patient41	False
Patient42	True
Patient43	True
Patient44	True
Patient45	True
Patient46	False
Patient47	True
Patient48	True
Patient49	True
Patient50	False
Patient51	False
Patient52	False

> **Note**
>
> Booleans represent one of two values: `True` or `False`. When you compare two values, the expression is evaluated and Python returns the Boolean answer:

It outputs a Boolean Series of the same length as the number of patients, with `True` for the patients that have a value greater than 2 on the 4th day, and `False` for the others. We can use this boolean array to slice the data and get only the values that are greater than 2:

```
patient_to_keep = data.iloc[:, 3] > 2
print(data[patient_to_keep])
```

```
           Day1  Day2  Day3  Day4  Day5  Day6  Day7  Day8  Day9  Day10  ...  \
Patient1      0     0     1     3     1     2     4     7     8      3  ...
Patient3      0     1     1     3     3     2     6     2     5      9  ...
Patient5      0     1     1     3     3     1     3     5     2      4  ...
Patient9      0     0     0     3     1     5     6     5     5      8  ...
Patient21     0     1     1     3     1     4     4     1     8      2  ...
Patient22     0     0     2     3     2     3     2     6     3      8  ...
Patient23     0     0     0     3     4     5     1     7     7      8  ...
Patient32     0     0     2     3     3     4     5     3     6      7  ...
Patient36     0     0     0     3     1     3     6     4     3      4  ...
Patient39     0     1     0     3     2     4     1     1     5      9  ...
Patient40     0     1     1     3     1     1     5     5     3      7  ...
Patient42     0     0     1     3     3     1     2     1     8      9  ...
Patient43     0     1     1     3     4     5     2     1     3      7  ...
Patient44     0     0     1     3     1     4     3     6     7      8  ...
Patient45     0     1     1     3     3     4     4     6     3      4  ...
Patient47     0     0     2     3     4     5     4     6     2      9  ...
Patient48     0     1     1     3     1     4     6     2     8      2  ...
Patient49     0     0     1     3     2     5     1     2     7      6  ...
Patient56     0     0     1     3     2     3     6     4     5      7  ...

           Day31  Day32  Day33  Day34  Day35  Day36  Day37  Day38  Day39  \
Patient1       4      4      5      7      3      4      2      3      0
Patient3      10      5      4      2      2      3      2      2      1
Patient5       9      6      3      2      2      4      2      0      1
Patient9       4      6      4      7      6      3      2      1      0
Patient21      3      2      4      3      1      5      4      2      2
Patient22      8      5      6      6      1      4      3      0      2
Patient23      4      4      8      2      6      5      1      0      1
Patient32      3      6      6      4      5      2      2      3      0
```

```
Patient36      3      9      5      1      6      5      4      2      2
Patient39      5      5      2      1      1      1      1      3      0
Patient40      2      3      6      3      3      5      4      3      2
Patient42      4      8      2      6      6      4      2      2      0
Patient43      5      8      5      5      6      1      2      1      2
Patient44     10      2      5      1      5      4      2      1      0
Patient45     10      6      8      7      2      5      4      3      1
Patient47      6      7      6      5      1      3      1      0      0
Patient48      6      9      5      6      1      1      2      1      2
Patient49     10      7      6      3      1      5      4      3      0
Patient56      3      5      3      5      4      5      3      3      0

           Day40
Patient1       0
Patient3       1
Patient5       1
Patient9       0
Patient21      0
Patient22      0
Patient23      0
Patient32      0
Patient36      0
Patient39      1
Patient40      1
Patient42      0
Patient43      0
Patient44      1
Patient45      1
Patient47      0
Patient48      1
Patient49      0
Patient56      1

[19 rows x 40 columns]
```

Rows of the DataFrame are being filtered by boolean values. If `True` the row is kept, if `False` it is dropped.

> **Note**
>
> The syntax does not use `.iloc[]` or `.loc[]` because we are not slicing with integer indices or labels, but with a boolean array. The boolean array is used to filter the rows of the

data, and all columns are included in the output.

Warning

The boolean array needs to be the same length as the number of rows in the data, otherwise you will get an error. For example, if you try to filter with a boolean array that has only 10 values, you will get an error because it does not match the number of patients (60).

```
patient_to_keep = data.iloc[:10, 3] > 2
print(data[patient_to_keep])
```

```
/tmp/ipykernel_2881/186501073.py:2: UserWarning: Boolean Series key will be reindexed to ma
  print(data[patient_to_keep])

IndexingError: Unalignable boolean Series provided as indexer (index of the boolean Series
---------------------------------------------------------------------------
IndexingError                             Traceback (most recent call last)
Cell In[53], line 2
      1 patient_to_keep = data.iloc[:10, 3] > 2
----> 2 print(data[patient_to_keep])
File /opt/hostedtoolcache/Python/3.10.20/x64/lib/python3.10/site-packages/pandas/core/frame
   3796 # Do we have a (boolean) 1d indexer?
   3797 if com.is_bool_indexer(key):
-> 3798     return self._getitem_bool_array(key)
   3800 # We are left with two options: a single key, and a collection of keys,
   3801 # We interpret tuples as collections only for non-MultiIndex
   3802 is_single_key = isinstance(key, tuple) or not is_list_like(key)
File /opt/hostedtoolcache/Python/3.10.20/x64/lib/python3.10/site-packages/pandas/core/frame
   3845     raise ValueError(
   3846         f"Item wrong length {len(key)} instead of {len(self.index)}."
   3847     )
   3849 # check_bool_indexer will throw exception if Series key cannot
   3850 # be reindexed to match DataFrame rows
-> 3851 key = check_bool_indexer(self.index, key)
   3852 indexer = key.nonzero()[0]
   3853 return self._take_with_is_copy(indexer, axis=0)
File /opt/hostedtoolcache/Python/3.10.20/x64/lib/python3.10/site-packages/pandas/core/index
   2550 indexer = result.index.get_indexer_for(index)
   2551 if -1 in indexer:
-> 2552     raise IndexingError(
```

```
    2553         "Unalignable boolean Series provided as "
    2554         "indexer (index of the boolean Series and of "
    2555         "the indexed object do not match)."
    2556     )
    2558 result = result.take(indexer)
    2560 # fall through for boolean
IndexingError: Unalignable boolean Series provided as indexer (index of the boolean Series
```

4.8 Analysing data

Pandas has several useful functions that take an array as input to perform operations on its values. If we want to find the average inflammation for all patients across days, for example, we can run:

```
print(data.mean())
```

```
Day1      0.000000
Day2      0.450000
Day3      1.116667
Day4      1.750000
Day5      2.433333
Day6      3.150000
Day7      3.800000
Day8      3.883333
Day9      5.233333
Day10     5.516667
Day11     5.950000
Day12     5.900000
Day13     8.350000
Day14     7.733333
Day15     8.366667
Day16     9.500000
Day17     9.583333
Day18    10.633333
Day19    11.566667
Day20    12.350000
Day21    13.250000
Day22    11.966667
Day23    11.033333
Day24    10.166667
```

```
Day25    10.000000
Day26     8.666667
Day27     9.150000
Day28     7.250000
Day29     7.333333
Day30     6.583333
Day31     6.066667
Day32     5.950000
Day33     5.116667
Day34     3.600000
Day35     3.300000
Day36     3.566667
Day37     2.483333
Day38     1.500000
Day39     1.133333
Day40     0.566667
dtype: float64
```

If we cant to find the average inflammation for all days across patients, we can run:

```
print(data.mean(axis=1))
```

```
Patient1     5.450
Patient2     5.425
Patient3     6.100
Patient4     5.900
Patient5     5.550
Patient6     6.225
Patient7     5.975
Patient8     6.650
Patient9     6.625
Patient10    6.525
Patient11    6.775
Patient12    5.800
Patient13    6.225
Patient14    5.750
Patient15    5.225
Patient16    6.300
Patient17    6.550
Patient18    5.700
Patient19    5.850
Patient20    6.550
```

```
Patient21    5.775
Patient22    5.825
Patient23    6.175
Patient24    6.100
Patient25    5.800
Patient26    6.425
Patient27    6.050
Patient28    6.025
Patient29    6.175
Patient30    6.550
Patient31    6.175
Patient32    6.350
Patient33    6.725
Patient34    6.125
Patient35    7.075
Patient36    5.725
Patient37    5.925
Patient38    6.150
Patient39    6.075
Patient40    5.750
Patient41    5.975
Patient42    5.725
Patient43    6.300
Patient44    5.900
Patient45    6.750
Patient46    5.925
Patient47    7.225
Patient48    6.150
Patient49    5.950
Patient50    6.275
Patient51    5.700
Patient52    6.100
Patient53    6.825
Patient54    5.975
Patient55    6.725
Patient56    5.700
Patient57    6.250
Patient58    6.400
Patient59    7.050
Patient60    5.900
dtype: float64
```

`axis` is an optional parameter of the method `.mean()` that specifies if the mean should be

calculated column-wise (i.e. day-wise, `axis=0`) or row-wise (i.e. patient-wise, `axis=1`). Same goes with `.median()`:

```
print(data.median())
```

```
Day1      0.0
Day2      0.0
Day3      1.0
Day4      2.0
Day5      2.0
Day6      3.0
Day7      4.0
Day8      4.0
Day9      5.0
Day10     6.0
Day11     6.0
Day12     5.5
Day13     9.5
Day14     8.0
Day15     8.0
Day16    10.0
Day17     8.5
Day18    11.0
Day19    11.5
Day20    13.0
Day21    14.0
Day22    13.0
Day23    11.0
Day24    10.0
Day25    10.5
Day26     9.0
Day27    10.0
Day28     7.0
Day29     7.0
Day30     7.0
Day31     6.0
Day32     6.0
Day33     5.0
Day34     4.0
Day35     3.0
Day36     4.0
Day37     2.0
```

```
Day38     1.0
Day39     1.0
Day40     1.0
dtype: float64
```

We can also get the max, min across days or patients with `data.max()` and `data.min()`, and the standard deviation with `data.std()`.

```
print(data.max(axis=0))
print(data.max(axis=1))
print(data.min())
print(data.std())
```

```
Day1      0
Day2      1
Day3      2
Day4      3
Day5      4
Day6      5
Day7      6
Day8      7
Day9      8
Day10     9
Day11    10
Day12    11
Day13    12
Day14    13
Day15    14
Day16    15
Day17    16
Day18    17
Day19    18
Day20    19
Day21    20
Day22    19
Day23    18
Day24    17
Day25    16
Day26    15
Day27    14
Day28    13
Day29    12
```

```
Day30    11
Day31    10
Day32     9
Day33     8
Day34     7
Day35     6
Day36     5
Day37     4
Day38     3
Day39     2
Day40     1
dtype: int64
Patient1     18
Patient2     18
Patient3     19
Patient4     17
Patient5     17
Patient6     18
Patient7     17
Patient8     20
Patient9     17
Patient10    18
Patient11    18
Patient12    18
Patient13    17
Patient14    16
Patient15    17
Patient16    18
Patient17    19
Patient18    19
Patient19    17
Patient20    19
Patient21    19
Patient22    16
Patient23    17
Patient24    15
Patient25    17
Patient26    17
Patient27    18
Patient28    17
Patient29    20
Patient30    17
Patient31    16
```

```
Patient32    19
Patient33    15
Patient34    15
Patient35    19
Patient36    17
Patient37    16
Patient38    17
Patient39    19
Patient40    16
Patient41    18
Patient42    19
Patient43    16
Patient44    19
Patient45    18
Patient46    16
Patient47    19
Patient48    15
Patient49    16
Patient50    18
Patient51    14
Patient52    20
Patient53    17
Patient54    15
Patient55    17
Patient56    16
Patient57    17
Patient58    19
Patient59    18
Patient60    18
dtype: int64
Day1     0
Day2     0
Day3     0
Day4     0
Day5     1
Day6     1
Day7     1
Day8     1
Day9     2
Day10    2
Day11    2
Day12    2
Day13    3
```

```
Day14    3
Day15    3
Day16    3
Day17    4
Day18    5
Day19    5
Day20    5
Day21    5
Day22    4
Day23    4
Day24    4
Day25    4
Day26    3
Day27    3
Day28    3
Day29    3
Day30    2
Day31    2
Day32    2
Day33    2
Day34    1
Day35    1
Day36    1
Day37    1
Day38    0
Day39    0
Day40    0
dtype: int64
Day1     0.000000
Day2     0.501692
Day3     0.738566
Day4     1.067628
Day5     1.140423
Day6     1.387902
Day7     1.725187
Day8     1.966600
Day9     1.942972
Day10    2.281032
Day11    2.764331
Day12    2.529152
Day13    3.161340
Day14    3.118380
Day15    3.782057
```

```
Day16    4.056800
Day17    3.854523
Day18    3.569963
Day19    3.980319
Day20    4.539189
Day21    4.276958
Day22    4.587997
Day23    4.234510
Day24    4.134053
Day25    3.741657
Day26    3.639085
Day27    3.240763
Day28    2.802088
Day29    2.703837
Day30    3.222230
Day31    2.536958
Day32    2.126707
Day33    1.637398
Day34    1.796418
Day35    1.778401
Day36    1.394501
Day37    1.127344
Day38    1.171700
Day39    0.812334
Day40    0.499717
dtype: float64
```

Tip

If you are interested in learning more about functions available in a package, you can check out the its documentation. Pandas is very famous and has a extensive documentation, for example you could check out the "Getting started tutorials".

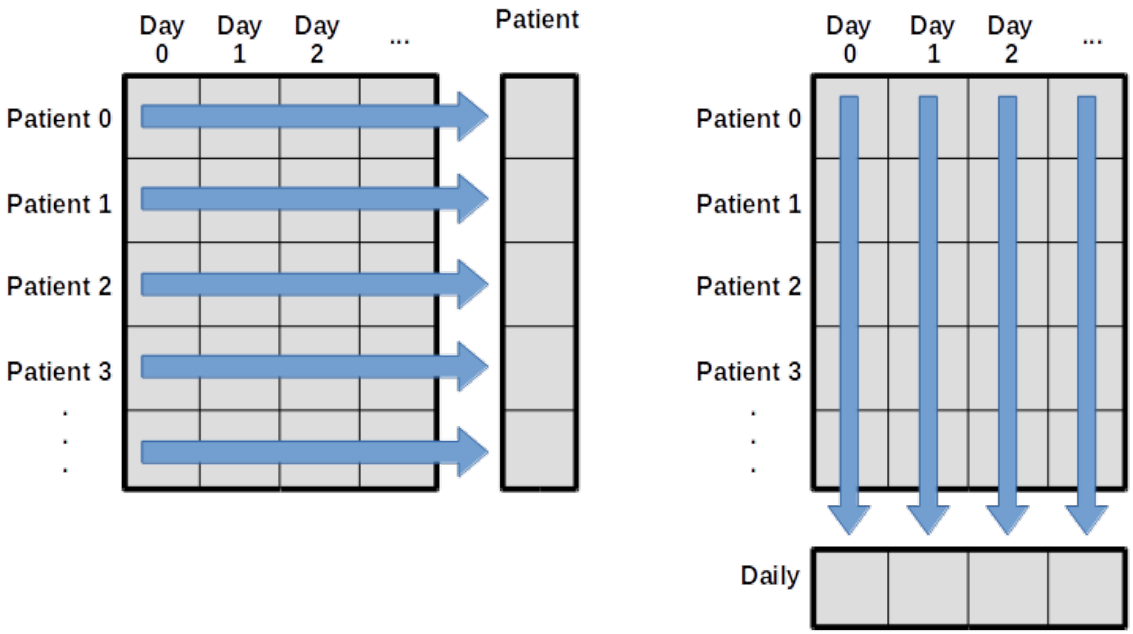


Figure 4.3: Schema of computing across rows or columns. .

4.9 Exercise

! Exercise

Output the data for the last 10 days of the data of the first 10 patients.
Output the minimum, maximum, and mean inflammation for the Day 20.
Compute the average inflammation for each days for the first 10 patients.
Filter the data to keep only the patients that have an average inflammation across days greater than 6.
Filter the data to keep only the patients that have a median inflammation across days greater than 6. Show only the last 10 days of the filtered data.

5 Visualizing data

5.1 Matplotlib package

A good way to develop insight is often to visualize data. We can explore a few features of Python's matplotlib library here. While there is no official plotting library, `matplotlib` is one of the standard packages to create visualizations in Python and is widely used in science.

> **Note**
>
> As for any package, we need to install it one on our computer to then be able to import it in our script and use its functions.
> Remember that installing a package is done outside of the python interpreter, in command line in a terminal.
>
> ```
> # In Linux/MacOS
> python -m pip install matplotlib
> # In Windows
> py -m pip install matplotlib
> ```

To shorten the name of the package when we call its functions, we can import it with a nickname, as follows:

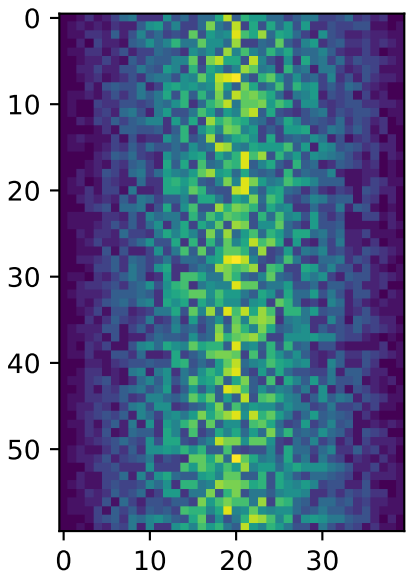
```
import pandas as pd

data = pd.read_csv('data/inflammation-01.csv', index_col=0)
```

For matplotlib, we usually import like so:

```
import matplotlib.pyplot as plt
```

`pyplot` is one of the modules of `matplotlib`. It contains functions to generate basic plots. We can display a heatmap of our data:

```
image = plt.imshow(data)
plt.show()
```

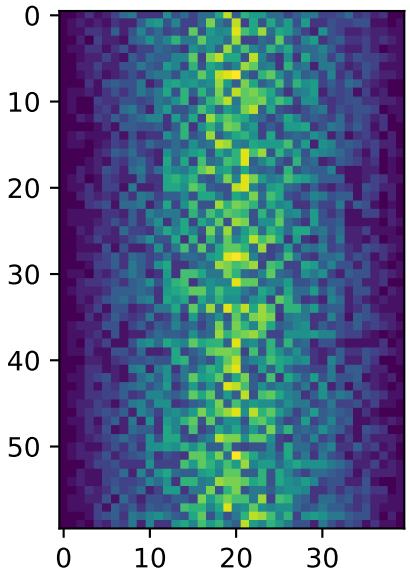


Each row in the heat map corresponds to a patient in the clinical trial dataset, and each column corresponds to a day in the dataset. Blue pixels in this heat map represent low values, while yellow pixels represent high values. As we can see, the general number of inflammation flare-ups for the patients rises and falls over a 40-day period. So far so good as this is in line with our knowledge of the clinical trial.

5.2 Function or object-oriented

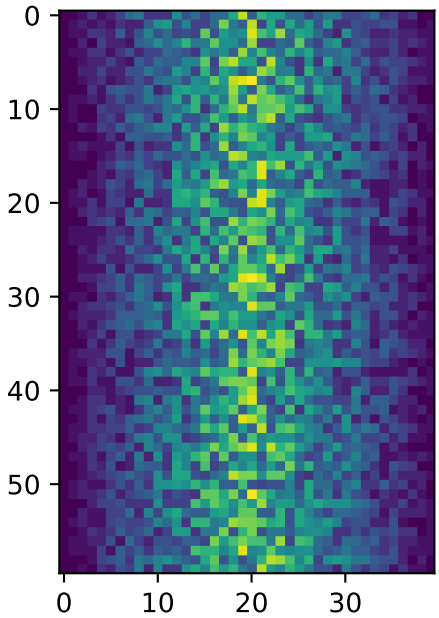
This first way of plotting is function-oriented. It relies on pyplot to implicitly create and manage the Figures and Axes, and use pyplot functions for plotting.

```
image = plt.imshow(data)
plt.show()
```



There is a second way of plotting called object-oriented. It needs to explicitly create Figures and Axes, and call methods on them (the "object-oriented (OO) style").

```
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)

ax.imshow(data)

fig.tight_layout()
plt.show()
```



You might encounter both styles of coding.

`fig` refers to the overall figure — the entire canvas that holds everything, including one or more plots. `ax` is the specific subplot (axes) where your data is drawn. In simple plots, we often interact only with `ax` to label axes or plot data. However, `fig` becomes useful when you want to set the overall figure title, adjust layout, or save the figure to a file.

The function `plt.figure()` creates a space into which we will place all of our plots. Each subplot is placed into the figure using its add_subplot method. The add_subplot method takes 3 parameters. The first `nrows` denotes how many total rows of subplots there are, the second parameter `ncols` refers to the total number of subplot columns, and the final parameter `index` denotes which subplot your variable is referencing (left-to-right, top-to-bottom).

> **Note**
>
> Notice that the names of the functions/methods called are not the same: the function `xlabel()` is used for the function-oriented manner and the method `set_xlabel()` is used for the object-oriented.

5.3 Matplotlib anatomy

Matplotlib graphs your data on Figures, each of which can contain one or more Axes. An Axes is an area where points can be specified in terms of x-y coordinates.

Axes contains a region for plotting data and includes generally two Axis objects (2D plots), a title, an x-label, and a y-label. The Axes methods (e.g. `.set_xlabel()`) are the primary interface for configuring most parts of your plot (adding data, controlling axis scales and limits, adding labels etc.).

An Axis sets the scale and limits and generate ticks (the marks on the Axis) and ticklabels (strings labeling the ticks).

> **Note**
>
> Be aware of the difference between Axes and Axis.

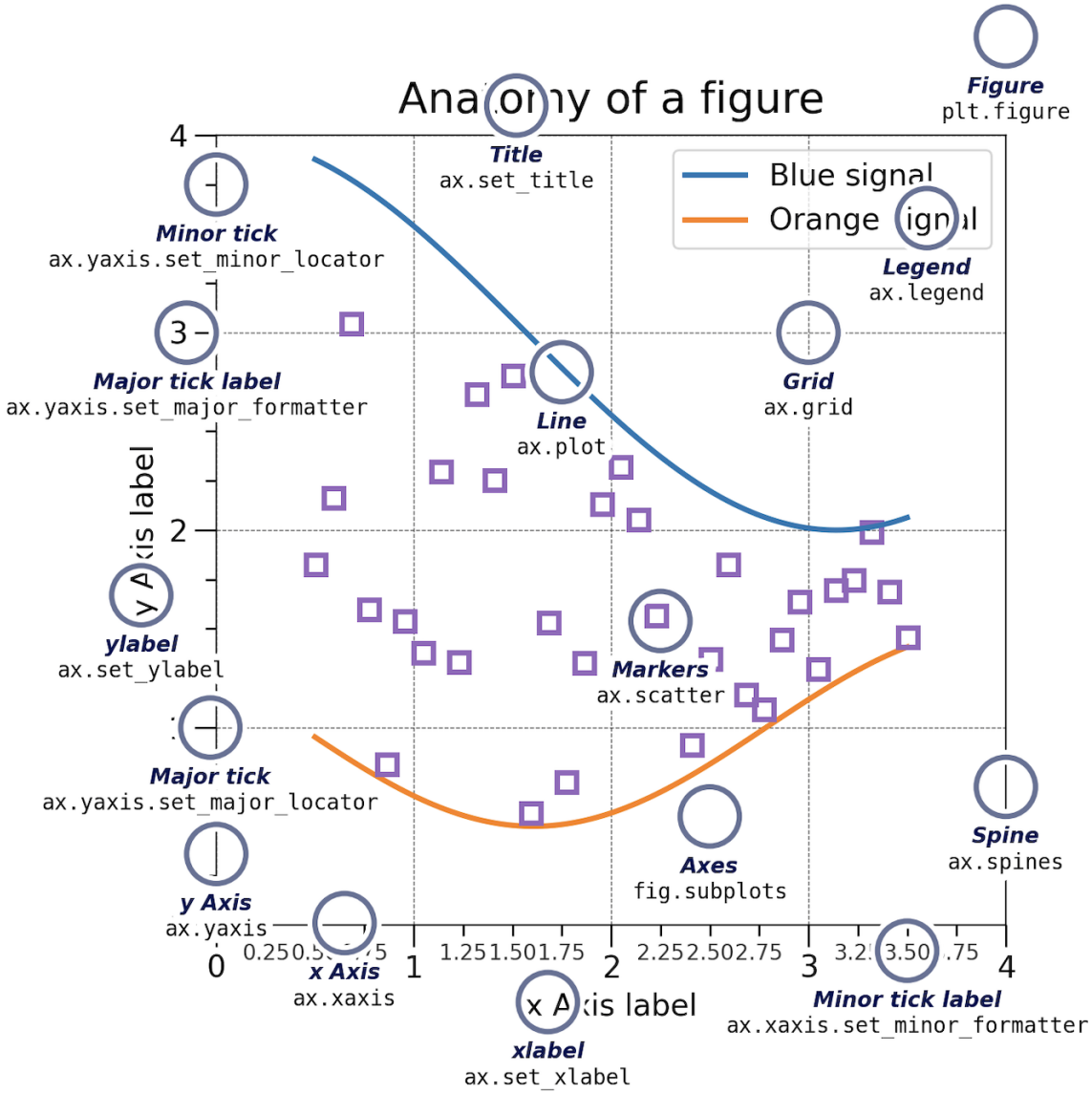


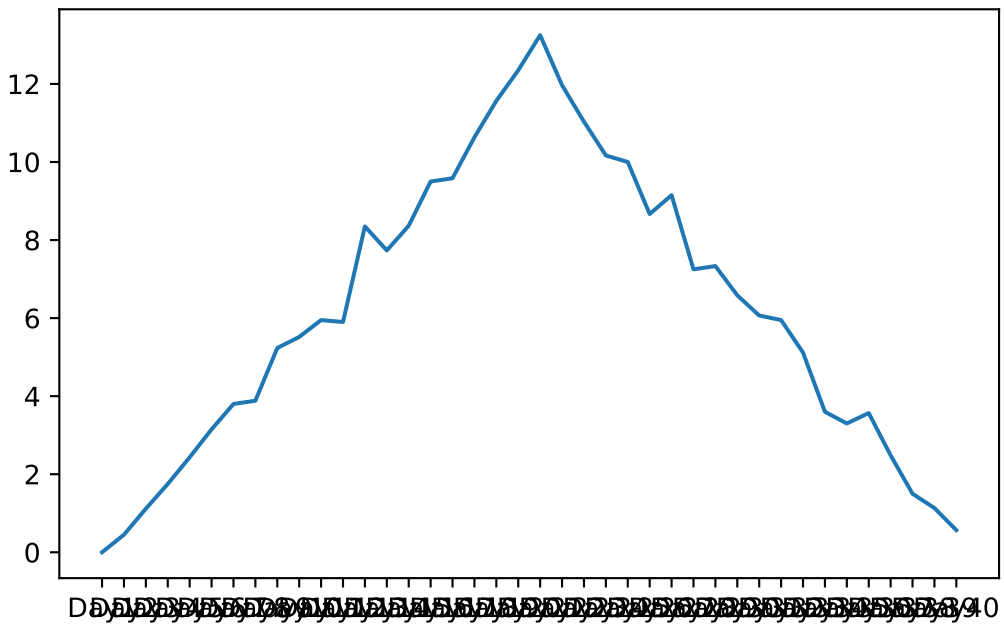
Figure 5.1: Anatomy of a matplotlib plot

There are many other plot available: `.plot()`, `.scatter()`, `.bar()`, `.hist()`, `.pie()`, `.boxplot()`…

Let's take a look at the average inflammation over time:

```
ave_inflammation = data.mean(axis=0)
```

```
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)

ax.plot(ave_inflammation)

fig.tight_layout()
plt.show()
```

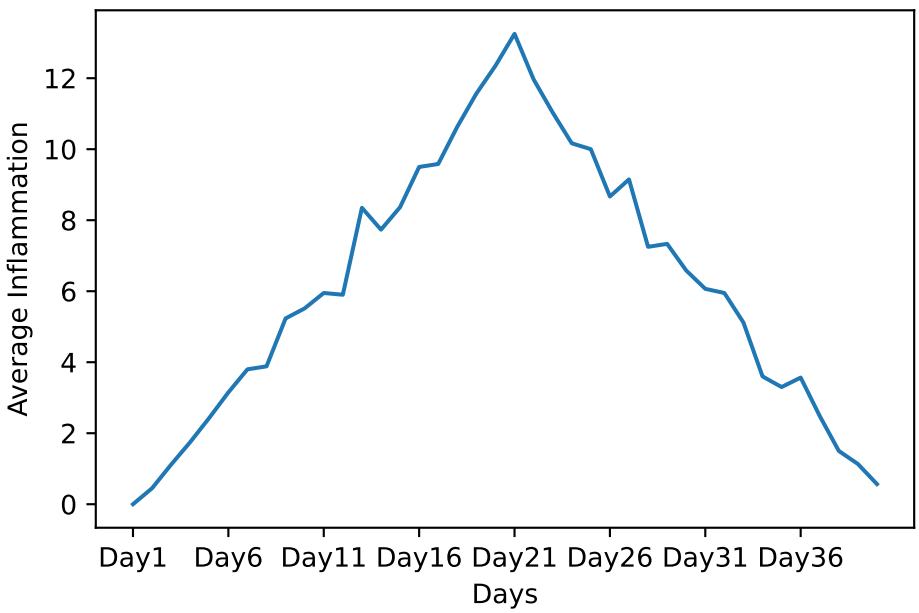


The x-axis of this plot represents the days of the clinical trial, while the y-axis represents the average inflammation level across all patients for each day. The plot shows a clear pattern of increasing inflammation levels over the first 20 days, followed by a decrease in inflammation levels over the remaining 20 days. This pattern is consistent with what we observed in the heat map and with our knowledge of the clinical trial.

Since our column names are `Day 1`, `Day 2`, etc., the x-axis of the plot is labeled with these column names. If we want to label the x-axis with the actual day numbers (1, 2, 3, etc.), we can modify the code as follows:

```
import numpy as np

ave_inflammation = data.mean(axis=0)

fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)
```

```
ax.plot(ave_inflammation)
ax.set_xlabel('Days')
ax.set_ylabel('Average Inflammation')
ax.set_xticks(np.arange(start=0, stop=40, step=5))

plt.show()
```



> **Note**
>
> For this solution we needed to import the numpy package, which is a fundamental package for scientific computing in Python. It provides support for arrays, matrices, and a large collection of mathematical functions to operate on these data structures.

Here, `np.arange()` is a function from the numpy package that generates an array of evenly spaced values within a specified range. In this case, `np.arange(start=0, stop=40, step=5)` generates an array of values starting from 0 up to (but not including) 40, with a step of 5. This means it will generate the values `[0, 5, 10, 15, 20, 25, 30, 35]`.

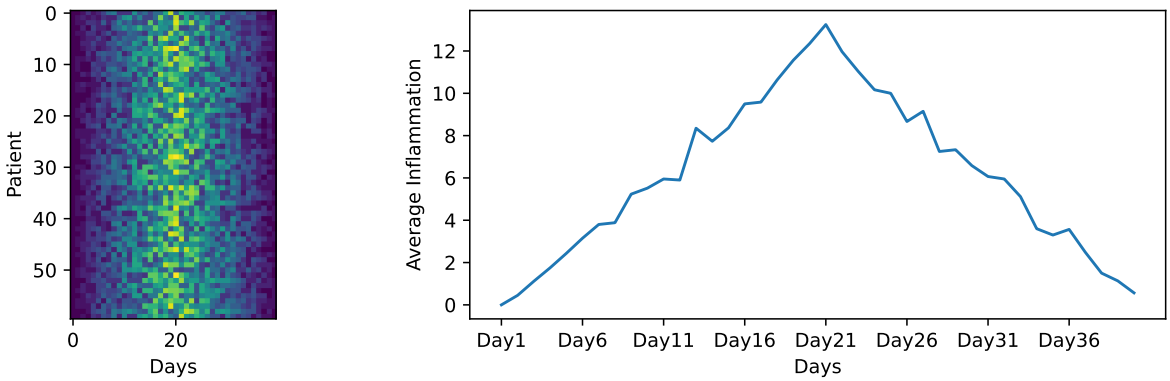
`ax.set_xticks()` expects a list of positions on the x-axis where the ticks should be placed. By passing the array generated by `np.arange()`, we are specifying that we want ticks at those positions (`0, 5, 10, 15, 20, 25, 30, 35`) on the x-axis of our plot. This allows us to label the x-axis with the actual day numbers corresponding to our data.

We also modified the labels of the x and y axes with `ax.set_xlabel()` and `ax.set_ylabel()` to make the plot more informative.

5.4 Grouping plots

You can group similar plots in a single figure using subplots. The parameter `figsize` tells Python how big to make this space. Each subplot is placed into the figure using `fig.add_subplot()`, which takes 3 parameters `nrows`, `ncols` and `index`. Each subplot is stored in a different variable (`axes1`, `axes2`, `axes3`). Once a subplot is created, the axes can be titled using the `ax.set_xlabel()` command (or `ax.set_ylabel()`). Here are our three plots side by side:

```
import matplotlib.pyplot as plt
import pandas as pd

data = pd.read_csv('data/inflammation-01.csv', index_col=0)

fig = plt.figure(figsize=(10.0, 3.0))

axes1 = fig.add_subplot(1, 2, 1)
axes2 = fig.add_subplot(1, 2, 2)

axes1.set_xlabel('Days')
axes1.set_ylabel('Patient')
axes1.imshow(data)

ave_inflammation = data.mean(axis=0)
axes2.set_xlabel('Days')
axes2.set_ylabel('Average Inflammation')
axes2.plot(ave_inflammation)
axes2.set_xticks(np.arange(start=0, stop=40, step=5))

fig.tight_layout()

plt.show()
```



5.5 Save a figure

You can save a figure with the `fig.savefig()` method, which takes as input the name of the file you want to save the figure to. The file will be saved in the current working directory, so make sure to provide the correct path if you want to save it somewhere else.

```
fig.savefig('data/figure.png')
```

> **Note**
>
> One could also run:
>
> ```
> plt.savefig('data/figure.png')
> ```
>
> ```
> <Figure size 1650x1050 with 0 Axes>
> ```
>
> The `matplotlib.pyplot` module works by automatically referencing the current active figure (i.e., the most recently created or interacted-with figure).
> But be careful, after a figure has been displayed to the screen (e.g. with `plt.show()`) matplotlib will make this variable refer to a new empty figure. Therefore, make sure you call `plt.savefig()` before the plot is displayed to the screen, otherwise you may find a file with an empty plot.

The plot can also be save as ps, pdf or svg. Moreover, the resolution can be modified. See the documentation of `.savefig()` for more parameters.

5.6 Matplotlib documentation

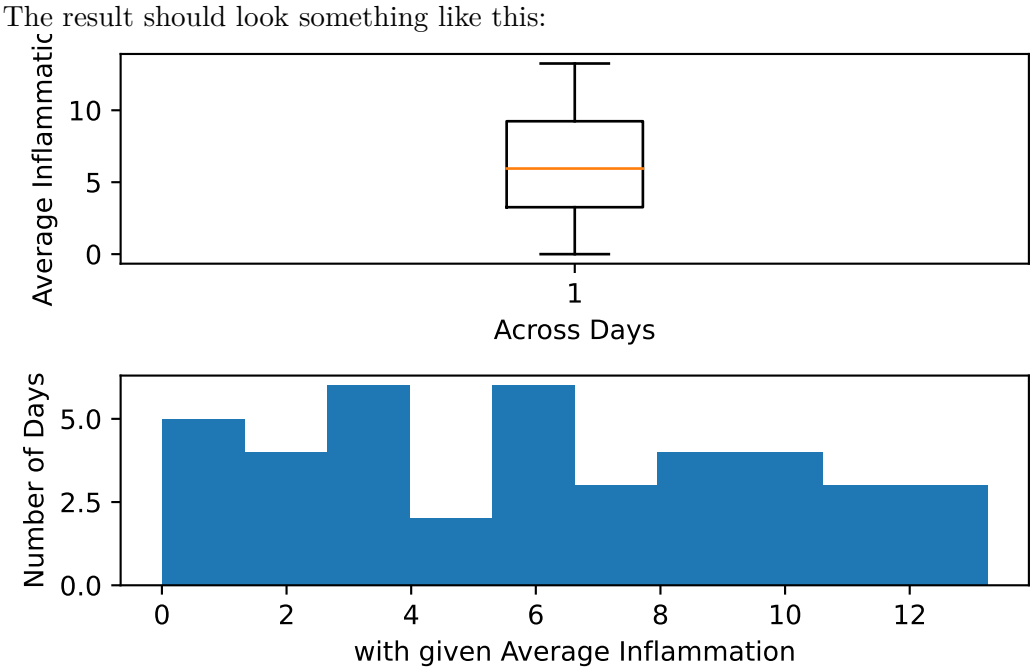
For more information, check out the following ressources:

- the documentation
- the cheat sheet
- any useful tutorial
- some inspiration

5.7 Exercise

! Exercise

Create a figure with two subplots, the one on the left being a boxplot (`.boxplot()`) of the average inflammation, and the one on the right being a histogram (`.hist()`) of the average inflammation. Make them one on top of one another instead of side by side. You can also set the x and y labels of the two plots to make them more informative.
The result should look something like this:



6 Storing multiple values in lists

We were provided with 9 more trial data, and we want to load and explore it as well.

Our goal now is to process all the inflammation data we have, which means that we still have eleven more files to go!

The natural first step is to collect the names of all the files that we have to process. In Python, a list is a way to store multiple values together. In this episode, we will learn how to store multiple values in a list as well as how to work with lists.

6.1 Creating a list

Data structures are a collection of data types (e.g. numerical, characters) and/or data structures, organized in some way. Lists are one of the data structures in Python. A list is a collection which is ordered and changeable. It allows duplicate members. They are created using square brackets `[]`.

We create a list by putting values inside square brackets and separating the values with commas:

```
odds = ['one', 3, 5, 7]
print('odds are:', odds)
```

```
odds are: ['one', 3, 5, 7]
```

> **Note**
>
> Notice that lists can contain elements of different types, here strings and integers.

We can access elements of a list using indices, i.e. numbered positions of elements in the list. The first item has index `[0]`, the second item has index `[1]` etc.

```
print('first element:', odds[0])
print('last element:', odds[3])
print('"-1" element:', odds[-1])
```

```
first element: one
last element: 7
"-1" element: 7
```

> **Tip**
>
> You can count backwards, with the index `[-1]` that retrieves the last item, `[-2]` the second to last, and so on. Because of this, `odds[3]` and `odds[-1]` point to the same element here.

Subsets of lists and strings can be accessed by specifying ranges of values in brackets, similar to how we accessed ranges of positions in a pandas DataFrame. This is commonly referred to as "slicing" the list/string.

```
binomial_name = 'Drosophila melanogaster'
group = binomial_name[0:10]
print('group:', group)

species = binomial_name[11:23]
print('species:', species)

chromosomes = ['X', 'Y', '2', '3', '4']
autosomes = chromosomes[2:5]
print('autosomes:', autosomes)
```

```
group: Drosophila
species: melanogaster
autosomes: ['2', '3', '4']
```

> **Tip**
>
> By leaving out the start value, the range will start at the first item:
>
> ```
> chromosomes[:2]
> ```
>
> ```
> ['X', 'Y']
> ```
>
> Similarly, by leaving out the end value, the range will end at the last item.
>
> ```
> chromosomes[2:]
> ```
>
> ```
> ['2', '3', '4']
> ```

> **Note**
>
> Remember, one way to recall how slices work is to think of the indices as pointing between characters, with the left edge of the first character numbered 0. Then the right edge of the last character of a string of n characters has index n, for example:
>
> ```
> +---+---+---+---+---+---+
> | P | y | t | h | o | n |
> +---+---+---+---+---+---+
> 0   1   2   3   4   5   6
> -6  -5  -4  -3  -2  -1
> ```
>
> The first row of numbers gives the position of the indices 0…6 in the string; the second row gives the corresponding negative indices. The slice from i to j consists of all characters between the edges labeled i and j, respectively.

You can get how many items are in a list with `len()`.

```
len(chromosomes)
```

```
5
```

6.2 Lists are mutable

There is one important difference between lists and strings: we can change the values in a list, but we cannot change individual characters in a string. For example:

```
names = ['Curie', 'Darwing', 'Turing']  # typo in Darwin's name
print('names is originally:', names)
names[1] = 'Darwin'  # correct the name
print('final value of names:', names)
```

```
names is originally: ['Curie', 'Darwing', 'Turing']
final value of names: ['Curie', 'Darwin', 'Turing']
```

works, but:

```
name = 'Darwin'
name[0] = 'd'
```

```
TypeError: 'str' object does not support item assignment
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[78], line 2
      1 name = 'Darwin'
----> 2 name[0] = 'd'
TypeError: 'str' object does not support item assignment
```

does not.

Data which can be modified in place is called mutable, while data which cannot be modified is called immutable. Strings and numbers are immutable. This does not mean that variables with string or number values are constants, but when we want to change the value of a string or number variable, we can only replace the old value with a completely new value.

Lists and pandas DataFrame, on the other hand, are mutable: we can modify them after they have been created. We can change individual elements, append new elements, or reorder the whole list. For some operations, like sorting, we can choose whether to use a function that modifies the data in-place or a function that returns a modified copy and leaves the original unchanged.

Be careful when modifying data in-place. If two variables refer to the same list, and you modify the list value, it will change for both variables!

```
seq = ['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'ACCT']
seq_mutated = seq # <-- seq and seq_mutated point to the *same* list data in memory
seq_mutated[-1] = 'AGGT'
print('sequences in seq:', seq)
print('sequences in seq_mutated:', seq_mutated)
```

```
sequences in seq: ['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'AGGT']
sequences in seq_mutated: ['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'AGGT']
```

If you want variables with mutable values to be independent, you must make a copy of the value when you assign it.

```
seq = ['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'ACCT']
seq_mutated = list(seq) # <-- makes a *copy* of the list
seq_mutated[-1] = 'AGGT'
print('sequences in seq:', seq)
print('sequences in seq_mutated:', seq_mutated)
```

```
sequences in seq: ['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'ACCT']
sequences in seq_mutated: ['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'AGGT']
```

> **Warning**
>
> Because of pitfalls like this, code which modifies data in place can be more difficult to understand. However, it is often far more efficient to modify a large data structure in place than to create a modified copy for every small change. You should consider both of these aspects when writing your code.

6.3 Nested lists

Since a list can contain any Python data types/structures, it can even contain other lists.

For example, you could represent sequences in a list of lists, where each inner list contains the sequences of one patient:

```
seqs = [['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'ACCT'],
['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'AGGT'],
['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'TCCA'],
['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'AGGT', 'ACCT']]
```

First, you can reference each row (i.e. patient):

```
print(seqs[0]) # sequences for first patient
```

```
['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'ACCT']
```

To reference a specific sequence, you can use two indices. The first index represents the patient (from top to bottom) and the second index represents the specific sequence (from left to right).

```
print(seqs[1][-1]) # last sequence for second patient
```

```
AGGT
```

You could also access a specific base of a sequence:

```
print(seqs[1][-1][0]) # first (0) base of last (-1) sequence for second (1) patient
```

```
A
```

6.4 Some list methods

There are many ways to change the content of lists besides assigning new values to individual elements:

```
odds.append(11)
print('odds after adding a value:', odds)
```

```
odds after adding a value: ['one', 3, 5, 7, 11]
```

```
removed_element = odds.pop(0)
print('odds after removing the first element:', odds)
print('removed_element:', removed_element)
```

```
odds after removing the first element: [3, 5, 7, 11]
removed_element: one
```

```
odds.reverse()
print('odds after reversing:', odds)
```

```
odds after reversing: [11, 7, 5, 3]
```

While modifying in place, it is useful to remember that Python treats lists in a slightly counter-intuitive way.

As we saw earlier, when we modified the `seq` list item in-place, if we make a list, (attempt to) copy it and then modify this list, we can cause all sorts of trouble. This also applies to modifying the list using the above functions:

```
odds = [3, 5, 7]
primes = odds
primes.append(2)
print('primes:', primes)
print('odds:', odds)
```

```
primes: [3, 5, 7, 2]
odds: [3, 5, 7, 2]
```

This is because Python stores a list in memory, and then can use multiple names to refer to the same list. If all we want to do is copy a (simple) list, we can again use the list function, so we do not modify a list we did not mean to:

```
odds = [3, 5, 7]
primes = list(odds)
primes.append(2)
print('primes:', primes)
print('odds:', odds)
```

```
primes: [3, 5, 7, 2]
odds: [3, 5, 7]
```

Here are a few methods for lists:

Method	Description
`.append()`	Inserts an item at the end
`.insert()`	Inserts an item at the specified index
`.extend()`	Append elements from another list to the current list
`.remove()`	Removes the first occurance of a specified item
`.pop()`	Removes the specified (by default last) index

You could also concatenate two lists with the `+` or `*` operator:

```
seqs[0] * 2
seqs[0] + seqs[1]
```

```
['ATGAAGGGTCCAAAA',
 'AGTCCCCGTATGAT',
 'ACCT',
 'ACCT',
 'ATGAAGGGTCCAAAA',
 'AGTCCCCGTATGAT',
 'ACCT',
 'AGGT']
```

6.5 Exercise

Exercise

1. Create a list `l = ['AAA', 'AAT', 'AAC']`, and add `AAG` at the end, using `.append()`.
2. Replace all `T` into `U` in the element `AAT`, using `.replace()`, which is a string method (documentation here or `help(str.replace)`).

Exercise

Use slicing to access only the last four characters of a string or entries of a list.

```
string_for_slicing = 'Observation date: 02-Feb-2013'
list_for_slicing = [['fluorine', 'F'],
                    ['chlorine', 'Cl'],
                    ['bromine', 'Br'],
                    ['iodine', 'I'],
                    ['astatine', 'At']]
```

Would your solution work regardless of whether you knew beforehand the length of the string or list (e.g. if you wanted to apply the solution to a set of lists of different lengths)? If not, try to change your approach to make it more robust.
Hint: Remember that indices can be negative as well as positive

7 Repeating actions with loops

We have access to ten data sets right now, and we will want to create the same plots for all of them. We could copy and paste the code we used to create the plot for the first data set, and change the name of the data variable each time, but that would be very inefficient and error-prone. We want to create plots for all of our data sets with a single statement. To do that, we'll have to teach the computer how to repeat things.

Before applying loops to our data sets, we will first learn how to use loops with simpler examples.

7.1 How to use loops

An example task that we might want to repeat is accessing numbers in a list, which we will do by printing each number on a line of its own.

```
odds = [1, 3, 5, 7]
```

In Python, a list is basically an ordered collection of elements, and every element has a unique number associated with it, its index. This means that we can access elements in a list using their indices. For example, we can get the first number in the list `odds`, by using `odds[0]`. One way to print each number is to use four print statements:

```
print(odds[0])
print(odds[1])
print(odds[2])
print(odds[3])
```

```
1
3
5
7
```

This is a bad approach for three reasons:

- Not scalable. Imagine you need to print a list that has hundreds of elements. You would have to write hundreds of print statements, which is not only inefficient but also very error-prone. You might forget to print some elements, or you might make a typo in the index of the element you want to print.
- Difficult to maintain. If we want to decorate each printed element with prefix, or any other character, we would have to change four lines of code. While this might not be a problem for small lists, it would definitely be a problem for longer ones.
- Fragile. If we use it with a list that has more elements than what we initially envisioned, it will only display part of the list's elements. A shorter list, on the other hand, will cause an error because it will be trying to display elements of the list that do not exist.

```
print(odds[0])
print(odds[1])
print(odds[2])
print(odds[3])
print(odds[4])
```

```
1
3
5
7

IndexError: list index out of range
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[97], line 5
      3 print(odds[2])
      4 print(odds[3])
----> 5 print(odds[4])
IndexError: list index out of range
```

Here's a better approach: a for loop

```
odds = [1, 3, 5, 7]
for num in odds:
    print(num)
```

```
1
3
5
7
```

What it does is the following: it processes each element in the list `odds`, called in the following code `num`, and prints it.

A for loop is an iteration. An iteration involves repeating a set of instructions or a block of code multiple times. Iterating through data structures like lists allows you to access each element individually, making it easier to perform operations on them.

When using a for loop, you iterate over a sequence of elements, such as a list. Here is the general syntax of a for loop in Python:

```
for item in data_structure:
    do task a
```

There must be a colon at the end of the line starting the loop, and we must indent anything we want to run inside the loop. Unlike many other languages, there is no command to signify the end of the loop body (e.g. `end for`); everything indented after the for statement belongs to the loop.

The loop will execute the indented block of code for each element in the sequence until all elements have been processed. This is particularly useful when you know the number of times you need to iterate.

Using the odds example above, the loop might look like this:

where each number (`num`) in the variable `odds` is looped through and printed one number after another. The other numbers in the diagram denote which loop cycle the number was printed in (1 being the first loop cycle, and 6 being the final loop cycle).

> **Note**
>
> We can call the loop variable (here `num`) anything we like.
>
> ```
> odds = [1, 3, 5, 7]
> for banana in odds:
>  print(banana)
> ```
>
> ```
> 1
> 3
> 5
> 7
> ```
>
> But it is a good idea to choose variable names that are meaningful, otherwise it would be more difficult to understand what the loop is doing.

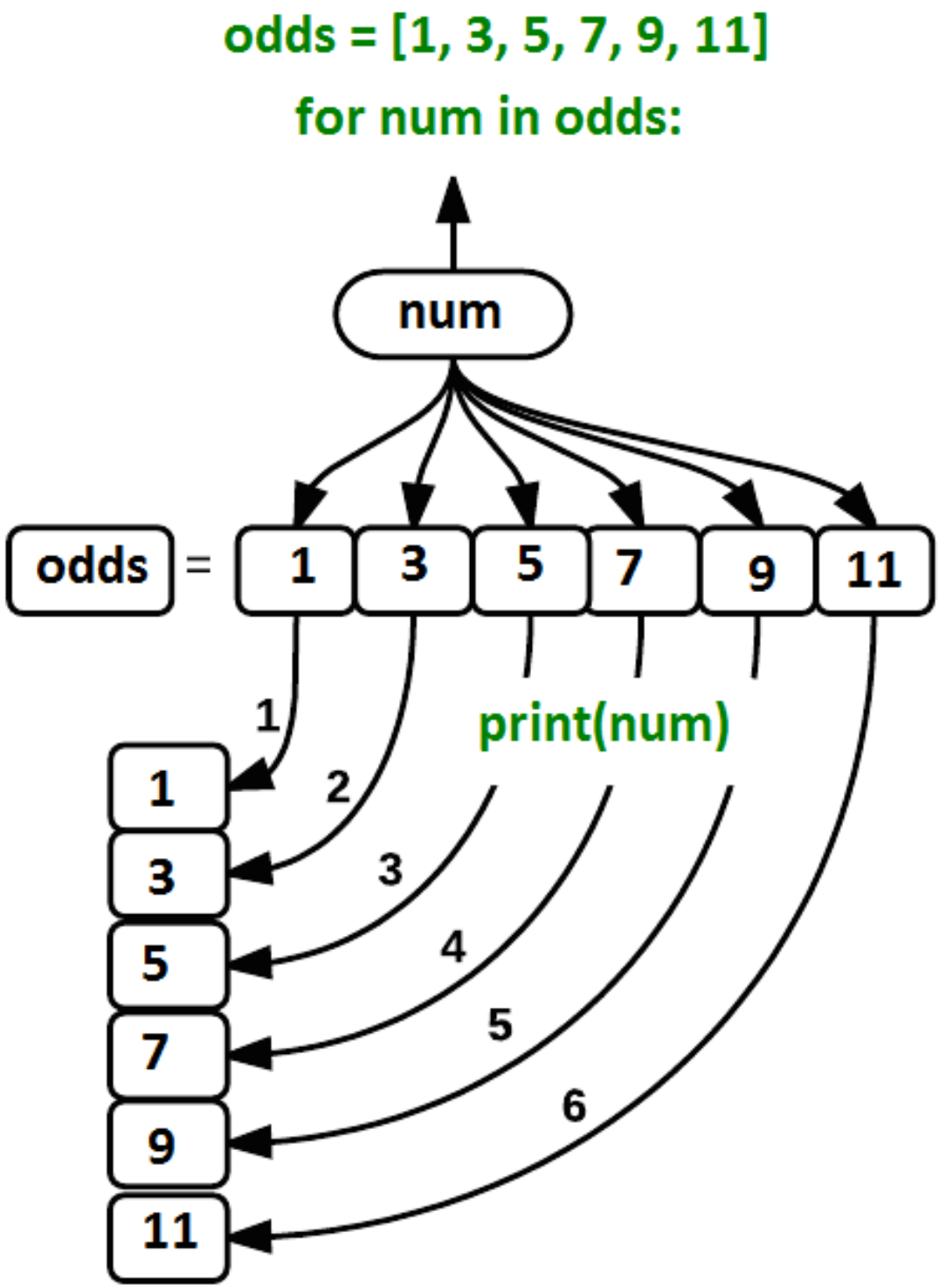


Figure 7.1: Loops schema

7.2 Notes on indentation

> **Note**
>
> Python relies on **indentation** (the spaces at the beginning of the lines).

Indentation is not just for readability. In Python, you use spaces or tabs to indent code blocks. Python uses it to determine the scope of functions, loops, conditional statements, and classes.

Any code that is at the same level of indentation is considered part of the same block. Blocks of code are typically defined by starting a line with a colon (`:`) and then indenting the following lines.

When you have nested structures like a loop inside another loop, you must further to show the hierarchy. Each level of indentation represents a deeper level of nesting.

It's essential to be consistent with your indentation throughout your code. The styling guide of Python PEP8 recommands 4 spaces as indentation.

> **Exercise**
>
> Here are two codes, they all are, can you tell why?
> Of course, you can run them and read the error that Python gives!

```
list_for_slicing = [['fluorine', 'F'],
                    ['chlorine', 'Cl'],
                    ['bromine', 'Br'],
                    ['iodine', 'I'],
                    ['astatine', 'At']]

for element in list_for_slicing:
  for subelement in element:
  print(subelement)
```

```
odds = [3, 5, 7]

for num in odds + odds
  print(num)
```

7.3 Loops and updating variables

Here's another loop that repeatedly updates a variable:

```
length = 0
names = ['Curie', 'Darwin', 'Turing']
for value in names:
    length = length + 1
print('There are', length, 'names in the list.')
```

```
There are 3 names in the list.
```

It's worth tracing the execution of this little program step by step. Since there are three names in `names`, the statement on line 4 will be executed three times. The first time around, `length` is `0` (the value assigned to it on line 1) and value is `Curie`. The statement adds 1 to the old value of `length`, producing `1`, and updates `length` to refer to that new value. The next time around, value is `Darwin` and `length` is `1`, so `length` is updated to be `2`. After one more update, `length` is `3`; since there is nothing left in names for Python to process, the loop finishes and the `print()` function on line 5 tells us our final answer.

Of course we could have just used `length(names)` to get the same answer, but this example is meant to illustrate how loops work, and how they can be used to update variables.

> **Note**
>
> Note that a loop variable is a variable that is being used to record progress in a loop. It still exists after the loop is over, and we can re-use variables previously defined as loop variables as well:
>
> ```
> name = 'Rosalind'
> for name in ['Curie', 'Darwin', 'Turing']:
>  print(name)
> print('after the loop, name is', name)
> ```
>
> ```
> Curie
> Darwin
> Turing
> after the loop, name is Turing
> ```

We can modify a variable using a loop, but we cannot modify an element of a list so easily.

Here is an example:

```python
all_codons = [
    'AAA', 'AAC', 'AAG', 'AAT',
    'ACA', 'ACC', 'ACG', 'ACT',
    'AGA', 'AGC', 'AGG', 'AGT',
    'ATA', 'ATC', 'ATG', 'ATT',
    'CAA', 'CAC', 'CAG', 'CAT',
    'CCA', 'CCC', 'CCG', 'CCT',
    'CGA', 'CGC', 'CGG', 'CGT',
    'CTA', 'CTC', 'CTG', 'CTT',
    'GAA', 'GAC', 'GAG', 'GAT',
    'GCA', 'GCC', 'GCG', 'GCT',
    'GGA', 'GGC', 'GGG', 'GGT',
    'GTA', 'GTC', 'GTG', 'GTT',
    'TAA', 'TAC', 'TAG', 'TAT',
    'TCA', 'TCC', 'TCG', 'TCT',
    'TGA', 'TGC', 'TGG', 'TGT',
    'TTA', 'TTC', 'TTG', 'TTT'
]

for codon in all_codons:
  codon = codon.replace('T', 'U')

print(all_codons)
```

```
['AAA', 'AAC', 'AAG', 'AAT', 'ACA', 'ACC', 'ACG', 'ACT', 'AGA', 'AGC', 'AGG', 'AGT', 'ATA',
```

This is `codon` is a copy of the item in a list, not a reference to it. So changing it does not do anything to the original list.

You would have to use an iterator.

7.4 Notes on iterators

An iterator is a special object that gives values in succession.

A way to modify the list would be to use an iterator to access the original data. The `range(start, stop, step)` function creates an iterator to count from one integer to another with a certain step (an optional parameter).

```
for i in range(2, 10, 1):
    print(i, end='  ')
```

```
2  3  4  5  6  7  8  9
```

We could count from 0 to the size of the list, loop though every element of the list by calling them by their index, and modify them if necessary. That's what the following code does:

```
for i in range(0, len(all_codons)):
  if 'T' in all_codons[i] :
    all_codons[i] = all_codons[i].replace('T', 'U')

print(all_codons)
```

```
['AAA', 'AAC', 'AAG', 'AAU', 'ACA', 'ACC', 'ACG', 'ACU', 'AGA', 'AGC', 'AGG', 'AGU', 'AUA',
```

Warning

A list is iterable but not an iterator. The difference is that they are reusable, see:

```
l = [1,2,3,4]

for i in l:
  print(i)

for i in l:
  print(i)
```

```
1
2
3
4
1
2
3
4
```

```
# Convert to iterable
il = iter(l)

for i in il:
  print(i)

for i in il:
  print(i)
print(all_codons)
```

```
1
2
3
4
['AAA', 'AAC', 'AAG', 'AAU', 'ACA', 'ACC', 'ACG', 'ACU', 'AGA', 'AGC', 'AGG', 'AGU', 'AUA',
```

Another useful function that returns an iterator is `enumerate()`. It is an iterator that generates pairs of index and value. It is commonly used when you need to access both the index and value of items simultaneously.

```
seq = 'ATGCATGC'

# Print index and identity of bases
for i, base in enumerate(seq):
    print(i, base)
```

```
0 A
1 T
2 G
3 C
4 A
5 T
6 G
7 C
```

7.5 Exercise

! Exercise

Write a loop that calculates the sum of elements in a list by adding each element and printing the final value, so feeding the loop with the list `[124, 402, 36]` shoud print `562` after summing up.

8 Analyzing data from multiple files

As a final piece to processing our inflammation data, we need a way to get a list of all the files in our `data` directory whose names start with `inflammation-` and end with `.csv`. The following library will help us to achieve this:

```
import glob
```

The `glob` library contains a function, also called `glob`, that finds files and directories whose names match a pattern. We provide those patterns as strings: the character `*` matches zero or more characters, while `?` matches any one character. We can use this to get the names of all the CSV files in the current directory:

```
print(glob.glob('data/inflammation*.csv'))
```

```
['data/inflammation-09.csv', 'data/inflammation-01.csv', 'data/inflammation-
08.csv', 'data/inflammation-07.csv', 'data/inflammation-11.csv', 'data/inflammation-
10.csv', 'data/inflammation-02.csv', 'data/inflammation-05.csv', 'data/inflammation-
04.csv', 'data/inflammation-06.csv', 'data/inflammation-03.csv', 'data/inflammation-
12.csv']
```

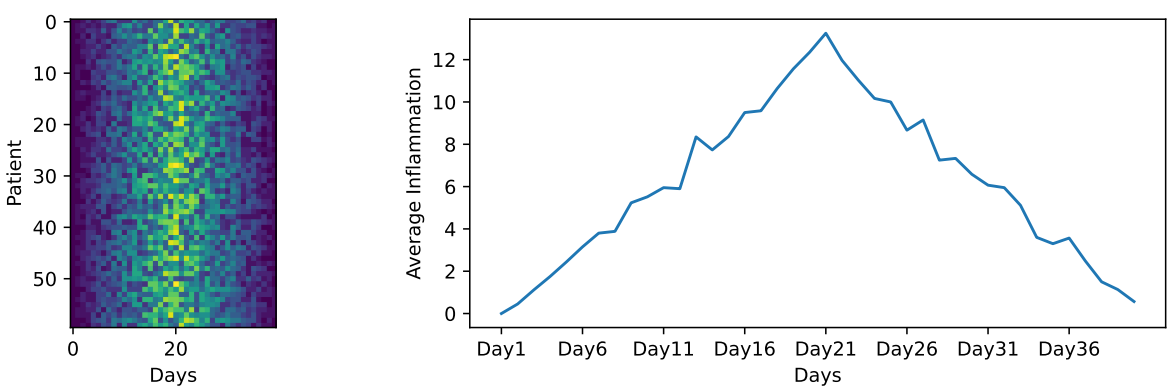
As these examples show, `glob.glob`'s result is a list of file and directory paths in arbitrary order. This means we can loop over it to do something with each filename in turn. In our case, the "something" we want to do is generate a set of plots for each file in our inflammation dataset.

If we want to start by analyzing just the first three files in alphabetical order, we can use the `sorted` built-in function to generate a new sorted list from the `glob.glob` output:

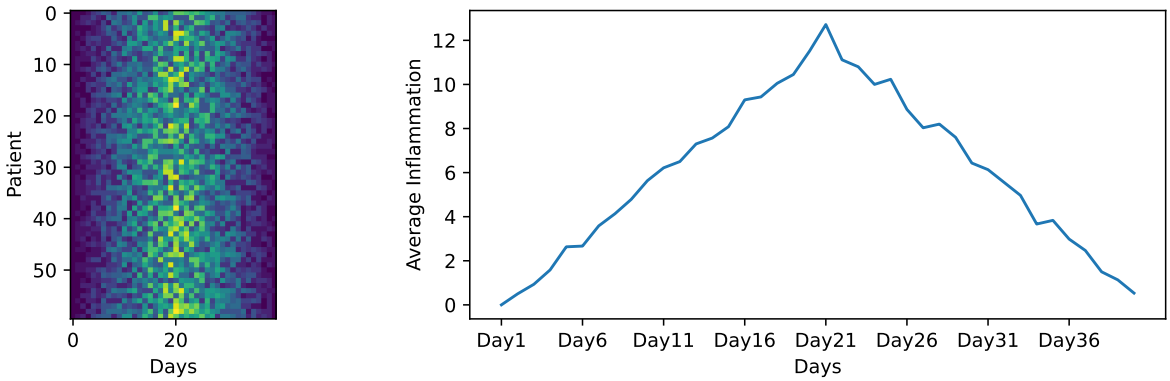
```
import glob
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

filenames = sorted(glob.glob('data/inflammation*.csv'))
filenames = filenames[0:3] # For sake of time, we will only analyze the first three files
```

```
for filename in filenames:
  print(filename)

  data = pd.read_csv(filename, index_col=0)

  fig = plt.figure(figsize=(10.0, 3.0))
  axes1 = fig.add_subplot(1, 2, 1)
  axes2 = fig.add_subplot(1, 2, 2)

  axes1.set_xlabel('Days')
  axes1.set_ylabel('Patient')
  axes1.imshow(data)

  ave_inflammation = data.mean(axis=0)
  axes2.set_xlabel('Days')
  axes2.set_ylabel('Average Inflammation')
  axes2.plot(ave_inflammation)
  axes2.set_xticks(np.arange(start=0, stop=40, step=5))

  fig.tight_layout()

  plt.show()
```

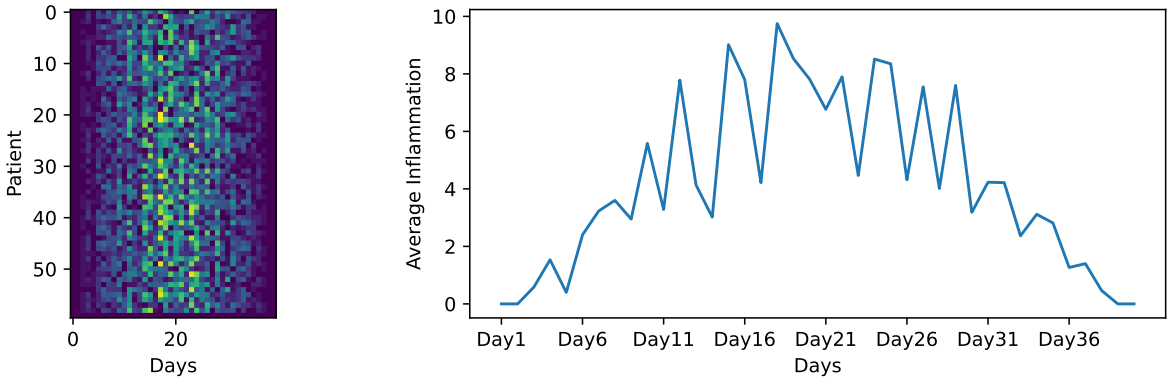
data/inflammation-01.csv



data/inflammation-02.csv



`data/inflammation-03.csv`



Here we are creating a loop that iterates through the list of filenames, and for each filename, it reads the data, creates a figure, and generates the two plots as we did before. This way, we can easily generate plots for all the files in our dataset without having to copy and paste code for each file.

This created 3 independent figures, one for each file.

> **❗ Exercise**
>
> We could also create one big figure with all the heatmap plots for all the files. What parameters of the `fig.add_subplot()` function would you change to do that? How should the loop be modified? Remember the `enumerate()` function that will be useful!

The 2 first plot show a similar trend, but the 3rd is different. If you look closely, in the 3rd heatmap, we can see that there are zero values sporadically distributed across all patients and days of the clinical trial, suggesting that there were potential issues with data collection throughout the trial. In addition, we can see that the last patient in the study didn't have any

inflammation flare-ups at all throughout the trial, suggesting that they may not even suffer from arthritis! Is there an issue with the data?

A good data miner does not only look at the mean to understand dataset…

8.1 Exercise

! Exercise

Plot the minimum and maximum inflammation for each day.
Try to plot it for one of the files, and then create a loop to plot it for all the files. Make use of the methods `.max()` and `min()`.
What is your conclusion on the trial data after looking at the heatmaps, mean, minimum and maximum inflammation?

9 Conclusion

Congrats! You now know the (very) basics of Python programming.

If you want to keep on practising with simple exercises, you can check out w3schools.

For more biology-related exercises check out pythonforbiologist.org, they have exercises availables in each chapters.

For french speakers, the AFPy (Association Francophone Python) has a learning tool called HackInScience.

Or keep on googling for more python exercises!

References

Here are some references and ressources that inspired this class :

- Python doc
- w3schools
- pythonforbiologists
- justinbois's Bootcamp
- Software carpentry 1
- Software carpentry 2

10 Lesson 2 - Conditional, Functions, User-input and Errors

11 Introduction

11.1 Aim of the class

At the end of this class, you will be able to:

- Use conditionals to make choices in your code
- Understand some major data structures in Python
- Create simple functions
- Run command-line scripts with user input
- Understand and raise exceptions

12 Making choices

In our last lesson, we discovered something suspicious was going on in our inflammation data by drawing some plots. How can we use Python to automatically recognize the different features we saw, and take a different action for each? In this lesson, we'll learn how to write code that runs only when certain conditions are true.

12.1 Conditionals

Conditionals allows you to make decisions in your code based on certain conditions.

```
if something is true:
    do task a
otherwise:
    do task b
```

For example we can ask Python to take different actions, depending on a condition, with an `if` statement:

```
num = 37
if num > 100:
    print('greater')
else:
    print('not greater')
print('done')
```

```
not greater
done
```

If the test that follows the `if` statement is true, the body of the `if` (i.e., the set of lines indented underneath it) is executed, and "greater" is printed. If the test is false, the body of the `else` is executed instead, and "not greater" is printed. Only one or the other is ever executed before continuing on with program execution to print "done".

Conditional statements don't have to include an `else`. If there isn't one, Python simply does nothing if the test is false:

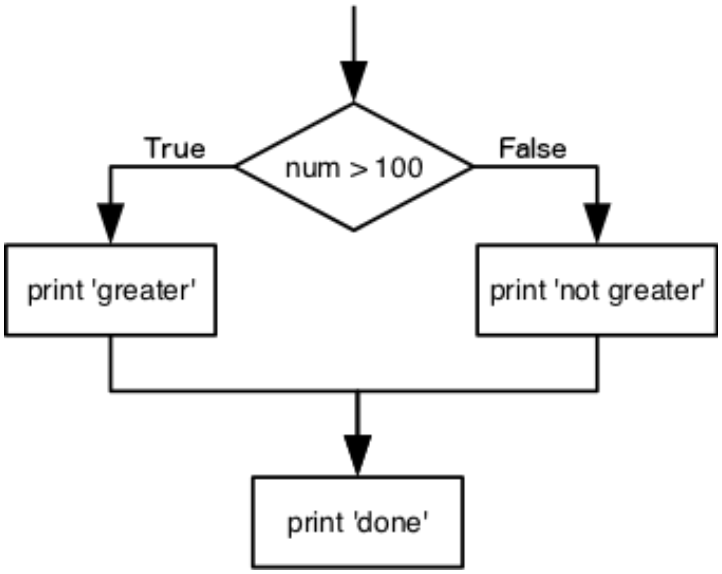


Figure 12.1: Conditional schema

```
num = 53
print('before conditional...')
if num > 100:
    print(num, 'is greater than 100')
print('...after conditional')
```

```
before conditional...
...after conditional
```

12.2 Comaparison and membership operators

Here we used a comparison operator (`>`) to compare the value of `num` to `100`. Here are other comparison operators:

Operator	Name
`==`	Equal
`!=`	Not equal
`>`	Greater than
`>=`	Greater than or equal to
`<`	Less than
`<=`	Less than or equal to

```
# Example
2 == 1 + 1
```

```
True
```

> **Note**
>
> Note that to test for equality we use a double equals sign `==` rather than a single equals sign `=` which is used to assign values.

> **Warning**
>
> You should never use equalty operators (`==`or `!=`) with floats or complex values.
>
> ```
> # Example
> 2.1 + 3.2 == 5.3
> ```
>
> ```
> False
> ```
>
> This is a floating point arithmetic problem seen in other programming languages. It is due to the difficulty of having a fixed number of binary digits (bits) to accurately represent some decimal number. This leads to small rounding errors in calculations.
>
> ```
> 2.1 + 3.2
> ```
>
> ```
> 5.300000000000001
> ```
>
> If you need to use equalty operators, do it with a degree of freedom:
>
> ```
> tol = 1e-6 ; abs((2.1 + 3.2) - 5.3) < tol
> ```
>
> ```
> True
> ```

The result of a comparison operator is a boolean value (`True` or `False`), which can be used in an `if` statement to control the flow of the program.

Booleans represent one of two values: `True` or `False`. When you compare two values, the expression is evaluated and Python returns the Boolean answer:

```
num = 37
print(num > 100)
```

```
False
```

There are also membership operators (`in` and `not in`). They are used to test if a value is present in a sequence (like a list or a string):

Operator	Description
`in`	Returns True if a sequence with the specified value is present in the object
`not in`	Returns True if a sequence with the specified value is not present in the object

```
seq = ['ATGAAGGGTCCAAAA', 'AGTCCCCGTATGAT', 'ACCT', 'ACCT']

print('ACCT' in seq)
print('G' in seq[-1])
```

```
True
False
```

12.3 `elif` statements

We can also chain several tests together using `elif` which is short for "else if". The `elif` keyword is Python's way of saying "if the previous conditions were not true, then try this condition". The following Python code uses `elif` to print the sign of a number.

```
num = -3

if num > 0:
    print(num, 'is positive')
elif num == 0:
    print(num, 'is zero')
else:
    print(num, 'is negative')
```

```
-3 is negative
```

We can also combine tests using logical operators `and`, `or` and `not` to create more complex conditions.

12.4 Logical operators

Logical operators are used to combine conditional statements:

Operator	Description
`and`	Returns True if both statements are true
`or`	Returns True if one of the statements is true
`not`	Reverse the result, returns False if the result is true

Result of the `and` oeprator:

Boolean 1	Boolean 2
`True`	`True`
`True`	`False`
`False`	`True`
`False`	`False`

```
# Example
False and False, False and True, True and False, True and True
```

```
(False, False, False, True)
```

Result of the `or` oeprator:

Boolean 1	Boolean 2
`True`	`True`
`True`	`False`
`False`	`True`
`False`	`False`

```
# Example
False or False, False or True, True or False, True or True
```

```
(False, True, True, True)
```

```
# Example
True or not True
```

```
True
```

```
if (1 > 0) and (-1 >= 0):
    print('both parts are true')
else:
    print('at least one part is false')
```

```
at least one part is false
```

```
if (1 < 0) or (1 >= 0):
    print('at least one test is true')
```

```
at least one test is true
```

12.5 Checking our data

Now that we've seen how conditionals work, we can use them to check for the suspicious features we saw in our inflammation data.

From the first couple of plots, we saw that maximum daily inflammation exhibits a strange behavior and raises one unit a day. Wouldn't it be a good idea to detect such behavior and report it as suspicious? Let's do that! However, instead of checking every single day of the study, let's merely check if maximum inflammation in the beginning (day 0) and in the middle (day 20) of the study are equal to the corresponding day numbers.

```
import pandas as pd

data = pd.read_csv('data/inflammation-01.csv', index_col=0)

max_inflammation_0 = data.max(axis=0).iloc[0]
max_inflammation_20 = data.max(axis=0).iloc[20]

if max_inflammation_0 == 0 and max_inflammation_20 == 20:
    print('Suspicious looking maxima!')
```

```
Suspicious looking maxima!
```

We also saw a different problem in the third dataset; the minima per day were all zero (looks like a healthy person snuck into our study). And if neither of these conditions are true, we can use else to give the all-clear:

```
data = pd.read_csv('data/inflammation-01.csv', index_col=0)

max_inflammation_0 = data.max(axis=0).iloc[0]
max_inflammation_20 = data.max(axis=0).iloc[20]

if max_inflammation_0 == 0 and max_inflammation_20 == 20:
    print('Suspicious looking maxima!')
elif data.min(axis=0).sum() == 0:
    print('Minima add up to zero!')
else:
    print('Seems OK!')
```

```
Suspicious looking maxima!
```

With another dataset:

```
data = pd.read_csv('data/inflammation-03.csv', index_col=0)

max_inflammation_0 = data.max(axis=0).iloc[0]
max_inflammation_20 = data.max(axis=0).iloc[20]

if max_inflammation_0 == 0 and max_inflammation_20 == 20:
    print('Suspicious looking maxima!')
elif data.min(axis=0).sum() == 0:
    print('Minima add up to zero!')
else:
    print('Seems OK!')
```

```
Minima add up to zero!
```

In this way, we have asked Python to do something different depending on the condition of our data. Here we printed messages in all cases, but we could also imagine not using the else catch-all so that messages are only printed when something is wrong, freeing us from having to manually examine every plot for features we've seen before.

12.6 Exercise

Exercise

With `gene1_expression` and `gene2_expression` given, are these 2 codes equivalent?

```
# Code A
if gene1_expression > gene2_expression:
  print("Gene 1 has higher expression level.")
elif gene1_expression < gene2_expression:
  print("Gene 2 has higher expression level.")
else:
  print("Gene 1 and Gene 2 have the same expression level.")
```

```
# Code B
if gene1_expression > gene2_expression:
  print("Gene 1 has higher expression level.")
else:
  if gene1_expression < gene2_expression:
    print("Gene 2 has higher expression level.")
  else:
    print("Gene 1 and Gene 2 have the same expression level.")
```

Exercise

Are these two codes equivalent?

```
# Code A
if "ATG" in dna_sequence:
  print("Start codon found.")
elif "TAG" in dna_sequence:
  print("Stop codon found.")
else:
  print("No interesting codon found.")
```

```
# Code B
if "ATG" in dna_sequence:
  print("Start codon found.")
  if "TAG" in dna_sequence:
    print("Stop codon found.")
else:
  print("No interesting codon found.")
```

! Exercise

This code is incorrect, can you tell why?

```
x = 7

if x > 5:
  print("x is greater than 5")
  if x > 10:
    print("x is greater than 10")
  elif x = 10:
    print("x equals 10")
  else:
    print("x is less than 10")
```

! Exercise

Instead of just printing if a file falls into one or the other category, we want to save in lists the name of the files that have suspicious maximum trend, a suspicious minimum trend and the ones that seem normal at the moment.
Modify the following code:

```
data = pd.read_csv('data/inflammation-03.csv', index_col=0)

max_inflammation_0 = data.max(axis=0).iloc[0]
max_inflammation_20 = data.max(axis=0).iloc[20]

if max_inflammation_0 == 0 and max_inflammation_20 == 20:
    print('Suspicious looking maxima!')
elif data.min(axis=0).sum() == 0:
    print('Minima add up to zero!')
else:
    print('Seems OK!')
```

```
Minima add up to zero!
```

You will need to loop through all the files in the `data` folder, and create three lists `maxima_suspicious_files`, `minima_suspicious_files` and `normal_files` to save the names of the files that fall into each category.
Remember to make use of the function `list.append(element)` to add an element to a list.

13 Storing multiple values in dictionaries

In the last exercise, we created three lists to store the names of the files that have suspicious maximum trend, a suspicious minimum trend and the ones that seem normal at the moment. We could also have stored this information in a dictionary, which is a data structure that allows us to store `key: value` pairs.

A dictionary is a collection which is ordered (as of Python >= 3.7), changeable and does not allow duplicates keys. Dictionaries are written with curly brackets `{}`, with keys and values. Keys must be unique and of an immutable type (like strings or numbers), while values can be of any type (like strings, numbers, lists, dictionnaries…) and can be duplicated.

```
organism1_genes = {
  #key: value;
  'BRCA1': 'DNA repair',
  'TP53': 'Tumor suppressor',
  'EGFR': 'Cell growth',
  'MYC': 'Regulation of gene expression'
}
```

Dictionary items can be referred to by using the key name.

```
organism1_genes["BRCA1"]
```

```
'DNA repair'
```

Dictionaries have specific methods. Here are a few:

Method	Description
`.items()`	Returns a list containing a tuple for each key value pair
`.keys()`	Returns a list containing the dictionary's keys
`.values()`	Returns a list of all the values in the dictionary
`.pop()`	Removes the element with the specified key

Method	Description
`.get()`	Returns the value of the specified key

! Exercise

Modify the previous code so that instead of having 3 lists `maxima_suspicious_files`, `minima_suspicious_files` and `normal_files` you have only one dictionnary, called `classify_files`. The values of the dictionary should be a list containing the name of the files, and the keys should be one of the following strings: `"suspicious maxima"`, `"suspicious minima"` or `"normal"`.
You can initialize the dictionnary like so:

```
categorize_files = {
    "suspicious maxima": [],
    "suspicious minima": [],
    "normal": []
}
```

! Exercise

From the dictionary `organism1_genes` created as example, get the value of the key `BRCA1`. If the key does not exist, return `Unknown` by default. Try your code before **and after** removing the `BRCA1` key:value pair.
Check the help of `get` by running `help(dict.get)`.

14 Creating functions

We have created a workflow to check if our data looks suspicious. In python we can wrap this workflow into a function, which is a reusable block of code that performs a specific task. Functions allow us to organize our code and make it more modular and easier to read.

For example, instead of reading a whole block of code, we could create a function called `check_suspicious_data()` that takes a filename as input and returns whether the data in the file is suspicious or not. But first let's learn how to create a function in Python.

> **Tip**
>
> A function usually takes some data as input (parameters that are required or optional), and usually returns an output (that can be of any type).
> We already learned how to run a predefined function in the last lesson. You need to write its name followed by parenthesis. Parameters are added inside the parenthesis as follow:
>
> ```
> # round(number, ndigits=None)
> x = round(number = 5.76543, ndigits = 2)
> print(x)
> ```
>
> ```
> 5.77
> ```
>
> To get more information about a function, use the `help()` function.
> We will now learn how to create our own function.

Imagine that we want to convert a temperature in Fahrenheit and converting it to Celsius. We could write:

```
fahrenheit_val = 99
celsius_val = ((fahrenheit_val - 32) * (5/9))
```

and for a second number we could just copy the line and rename the variables:

```
fahrenheit_val = 99
celsius_val = ((fahrenheit_val - 32) * (5/9))
```

```
fahrenheit_val2 = 43
celsius_val2 = ((fahrenheit_val2 - 32) * (5/9))
```

But we would be in trouble as soon as we had to do this more than a couple times. Cutting and pasting it is going to make our code get very long and very repetitive, very quickly. We'd like a way to package our code so that it is easier to reuse, a shorthand way of re-executing longer pieces of code. This is what functions are for.

14.1 Syntax

In python, a function is declared with the keyword `def` followed by its name, and the arguments inside parenthesis. The next block of code, corresponding to the content of the function, must be indented. The output is defined by the `return` keyword.

```
def explicit_fahr_to_celsius(temp):
    # Assign the converted value to a variable
    converted = ((temp - 32) * (5/9))
    # Return the value of the new variable
    return converted

def fahr_to_celsius(temp):
    # Return converted value more efficiently using the return
    # function without creating a new variable. This code does
    # the same thing as the previous function but it is more explicit
    # in explaining how the return command works.
    return ((temp - 32) * (5/9))
```

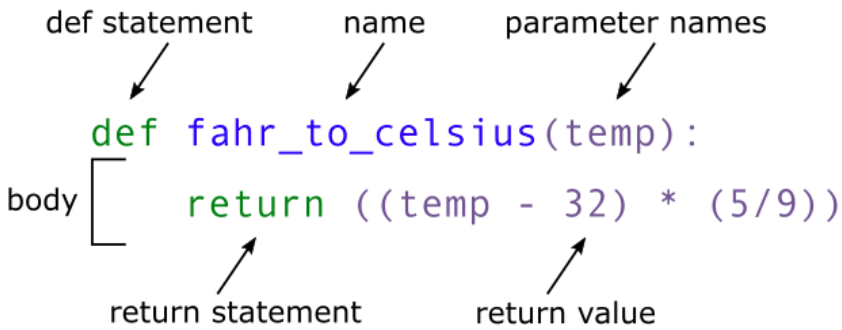


Figure 14.1: Function schema

When we call the function, the values we pass to it are assigned to those variables so that we can use them inside the function. Inside the function, we use a return statement to send a result back to whoever asked for it. Let's try running our function:

```
converted_temp = fahr_to_celsius(32)
print(converted_temp)

# or even,
print('freezing point of water:', fahr_to_celsius(32), 'C')
print('boiling point of water:', fahr_to_celsius(212), 'C')
```

```
0.0
freezing point of water: 0.0 C
boiling point of water: 100.0 C
```

We've successfully called the function that we defined, and we have access to the value that we returned.

14.2 Documentation

You can also add a description of the function directly after the function definition. It is the message that will be shown when running `help()`. As it can be along text over multiple lines, it is common to put it inside triple quotes `"""`.

```
def fahr_to_celsius(temp, inverted = False):
    """Returns the temperature in Celsius given a temperature in Fahrenheit,
    or the inverse (Celsius to Fahrenheit) if inversted is True.
    Parameters:
    temp (float): Temperature in Fahrenheit.
    inverted (bool, optional): Whether to convert from Celsius to Fahrenheit (True) or Fahre
    """
    return ((temp - 32) * (5/9))

help(fahr_to_celsius)
```

```
Help on function fahr_to_celsius in module __main__:

fahr_to_celsius(temp, inverted=False)
    Returns the temperature in Celsius given a temperature in Fahrenheit,
    or the inverse (Celsius to Fahrenheit) if inversted is True.
    Parameters:
    temp (float): Temperature in Fahrenheit.
    inverted (bool, optional): Whether to convert from Celsius to Fahrenheit (True) or Fahre
```

14.3 Arguments

You can have several arguments. They can be mandatory or optional. To make them optional, they need to have a default value assigned inside the function definition, like so:

```
def fahr_to_celsius(temp, inverted = False):
    """Returns the temperature in Celsius given a temperature in Fahrenheit,
    or the inverse (Celsius to Fahrenheit) if inversted is True.
    Parameters:
    temp (float): Temperature in Fahrenheit.
    inverted (bool, optional): Whether to convert from Celsius to Fahrenheit (True) or Fahren
    """
    if inverted:
        new_temp = ((temp * (9/5)) + 32)
    else:
        new_temp = ((temp - 32) * (5/9))
    return new_temp

# Or more efficiently:
def fahr_to_celsius(temp, inverted = False):
    """Returns the temperature in Celsius given a temperature in Fahrenheit,
    or the inverse (Celsius to Fahrenheit) if inversted is True.
    Parameters:
    temp (float): Temperature in Fahrenheit.
    inverted (bool, optional): Whether to convert from Celsius to Fahrenheit (True) or Fahren
    """
    if inverted:
        return ((temp * (9/5)) + 32)
    else:
        return ((temp - 32) * (5/9))

help(fahr_to_celsius)
```

```
Help on function fahr_to_celsius in module __main__:

fahr_to_celsius(temp, inverted=False)
    Returns the temperature in Celsius given a temperature in Fahrenheit,
    or the inverse (Celsius to Fahrenheit) if inversted is True.
    Parameters:
    temp (float): Temperature in Fahrenheit.
    inverted (bool, optional): Whether to convert from Celsius to Fahrenheit (True) or Fahren
```

> **Note**
>
> The `return` keyword is present twice, but the function will stop as soon as it reaches the first return statement, so if `inverted` is `True`, the second return statement will never be reached.

The input `temp` is mandatory but `inverted` is optional as it has a default value of `False`. If we call the function without providing a value for `inverted`, it will be set to `False` by default.

```
fahr_to_celsius(32)
```

```
0.0
```

```
fahr_to_celsius(inverted = True)
```

```
TypeError: fahr_to_celsius() missing 1 required positional argument: 'temp'
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[44], line 1
----> 1 fahr_to_celsius(inverted = True)
TypeError: fahr_to_celsius() missing 1 required positional argument: 'temp'
```

> **Note**
>
> Reminder: if you provide the parameters in the exact same order as they are defined, you don't have to name them. If you name the parameters you can switch their order. As good practice, put all required parameters first.
>
> ```
> fahr_to_celsius(inverted = True, temp = 100)
> ```
>
> ```
> 212.0
> ```
>
> ```
> fahr_to_celsius(100, True)
> ```
>
> ```
> 212.0
> ```

14.4 Output

If no `return` statement is given, then no output will be returned, but the function will be run.

```python
def fahr_to_celsius(temp, inverted = False):
    """Returns the temperature in Celsius given a temperature in Fahrenheit,
    or the inverse (Celsius to Fahrenheit) if inversted is True.
    Parameters:
    temp (float): Temperature in Fahrenheit.
    inverted (bool, optional): Whether to convert from Celsius to Fahrenheit (True) or Fahre
    """
    print("We are inside the function, the value of temp is:", temp)
    if inverted:
        new_temp = ((temp * (9/5)) + 32)
    else:
        new_temp = ((temp - 32) * (5/9))
```

```python
print(fahr_to_celsius(32))
```

```
We are inside the function, the value of temp is: 32
None
```

The output can be of any type. If you have a lot of things to return, you might want to return a `list`.

```python
def multiple_of_3(list_of_numbers):
  """Returns the number that are multiple of 3."""
  multiples = []
  for num in list_of_numbers:
    if num % 3 == 0:
      # num % 3 == 0 means that the remainder
      # of num divided by 3 is 0,
      # which means that num is a multiple of 3
      multiples.append(num)
  return multiples

multiple_of_3(range(1, 20, 2))
```

```
[3, 9, 15]
```

> **Note**
>
> This could be written as a one-liner.
>
> ```
> def multiple_of_3(list_of_numbers):
>  """Returns the number that are multiple of 3."""
>  multiples = [num for num in list_of_numbers if num % 3 == 0]
>  return multiples
>
> multiple_of_3(range(1, 20, 2))
> ```
>
> ```
> [3, 9, 15]
> ```

14.5 Variable Scope

In composing our temperature conversion function, we sometimes created a variable inside of the function, `new_temp`. We refer to such variables as local variables because they no longer exist once the function is done executing. If we try to access their values outside of the function, we will encounter an error:

```
print(new_temp)
```

```
NameError: name 'new_temp' is not defined
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[51], line 1
----> 1 print(new_temp)
NameError: name 'new_temp' is not defined
```

If you want to reuse the converted temperature having calculated it with `fahr_to_celsius`, you can store the result of the function call in a variable:

```
converted_temp = fahr_to_celsius(32)
print("Converted temp is:", converted_temp)
```

```
We are inside the function, the value of temp is: 32
Converted temp is: None
```

The variable `converted_temp`, being defined outside any function, is said to be global.

Interestingly, inside a function, one can read the value of such global variables:

```
def print_temperatures():
    print('temperature in Fahrenheit was:', temp_fahr)
    print('temperature in Celsius was:', temp_celsius)

temp_fahr = 212.0
temp_celsius = fahr_to_celsius(212.0)

print_temperatures()
```

```
We are inside the function, the value of temp is: 212.0
temperature in Fahrenheit was: 212.0
temperature in Celsius was: None
```

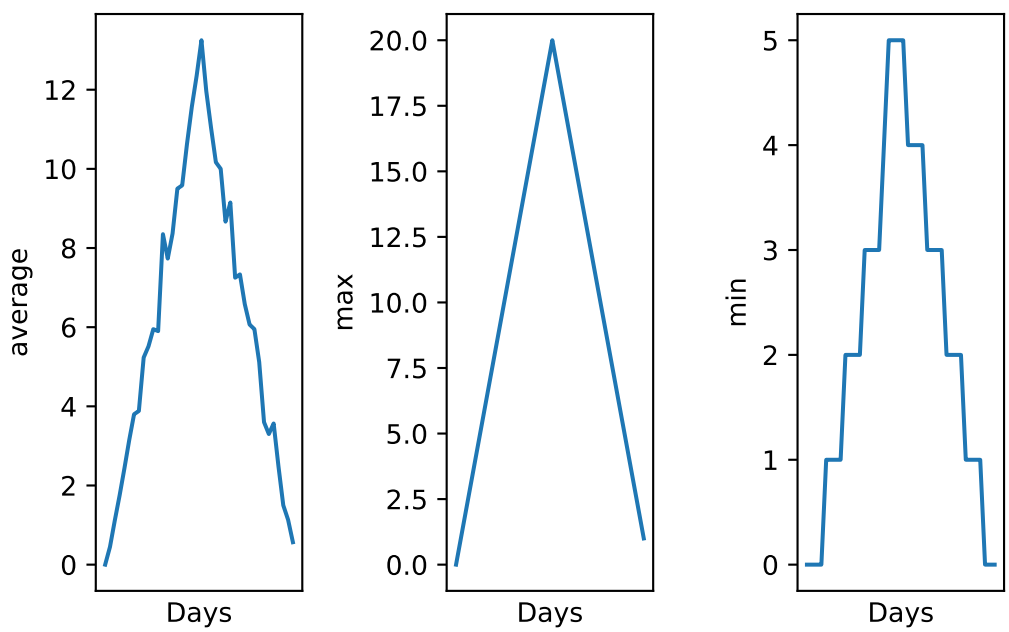
It is not recommended to modify the value of a global variable inside a function, as it can lead to unexpected behavior and make the code harder to debug. If you need to modify a global variable, it is better to return the modified value from the function and assign it to the global variable outside of the function.

14.6 Application on our data

Now that we know how to wrap bits of code up in functions, we can make our inflammation analysis easier to read and easier to reuse. First, let's make a `visualize()` function that generates our plots:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

def visualize(filename):
    """Visualizes the average, maximum and minimum inflammation per day for a given file.
    Parameters:
    filename (str): The path of the file to visualize.
    """
    data = pd.read_csv(filename, index_col=0)
    fig = plt.figure()

    axes1 = fig.add_subplot(1, 3, 1)
    axes2 = fig.add_subplot(1, 3, 2)
    axes3 = fig.add_subplot(1, 3, 3)
```

```
    axes1.set_ylabel('average')
    axes1.set_xlabel('Days')
    axes1.set_xticks([])
    axes1.plot(data.mean(axis=0))

    axes2.set_ylabel('max')
    axes2.set_xlabel('Days')
    axes2.set_xticks([])
    axes2.plot(data.max(axis=0))

    axes3.set_ylabel('min')
    axes3.plot(data.min(axis=0))
    axes3.set_xlabel('Days')
    axes3.set_xticks([])

    fig.tight_layout()
    plt.show()

visualize("data/inflammation-01.csv")
```



We did not use the `return` keyword. In Python, functions are not required to include a `return` statement and can be used for the sole purpose of grouping together pieces of code that conceptually do one thing. In such cases, function names usually describe what they do,

e.g. `visualize`. This is a common practice in Python, and it is not considered bad style. In fact, it can make the code more readable and easier to understand.

14.7 Exercise

! Exercise

Create a function that prints a message if the data in a file looks suspicious, using the same criteria as before (suspicious looking maxima and minima that add up to zero). No output needs to be returned, just print the message. It needs to also print the name of the file that is being checked.
Use it on one of the inflammation files, and then on all of them using a loop.
Add an optional parameter that asks the user to visulize the data if it has a suspicious maxima. If the parameter is set to `True`, the function should call the `visualize()` function we created earlier to show the mean, max and min plots of the data.

15 Command-Line programs

Sooner or later we will want to use our program in a pipeline or run it in a shell script to process thousands of data files. In order to do that in an efficient way, we need to make our programs work like other Unix command-line tools. For example, we may want a program that reads a dataset and prints the average inflammation per patient.

If you remember from the first lesson, it can be used either interactively in the terminal, or by writing a shell script (a file that contains shell commands) and running it.

> **Note**
>
> In this lesson we are switching from typing commands in a Python interpreter to typing commands in a shell terminal window (such as bash). When you see a `$` in front of a command that tells you to run that command in the shell rather than the Python interpreter.

15.1 Creating a python script

For this we need to create a Python script, which is a file that contains Python code. We can create a Python script using any text editor. The file should have a `.py` extension to indicate that it is a Python script.

Open a new file in your text editor, and write the following code:

```
#!/usr/bin/env python3

import pandas as pd

def main(filename, min = True, mean = True, max = True):
    data = pd.read_csv(filename, index_col=0)
    stats = {}

    if min:
        stats['min'] = data.min(axis=0).min()
    if mean:
```

```
        stats['mean'] = data.mean(axis=0).mean()
    if max:
        stats['max'] = data.max(axis=0).max()

    for key, value in stats.items():
        print(f"{key}: {value}")
```

Save it under the name `inflammation_stats.py`, in `~/Desktop/swc-python/code`.

> **Note**
>
> F-strings are a way to format strings in Python. They are defined by prefixing a string with the letter `f` or `F`, and they allow you to include expressions inside curly braces `{}` that will be evaluated at runtime and included in the string. They were introduced in Python 3.6 and provide a more concise and readable way to format strings. See the official documentation for more information on hwo to format strings.

You should already be in the `~/Desktop/swc-python/` directory. If not you can navigate to it using the `cd` command in the terminal. Then, you can run the script using the following command:

```
cd ~/Desktop/swc-python
```

You can verify you are in the right directory by running `pwd` (print working directory):

```
pwd
```

You can now run the python script:

```
./code/inflammation_stats.py
```

It should not output anything, as we only provided a function but did not call it. We need to call the `main()` function and provide it with the name of the file we want to analyze. We can do that by adding the following lines at the end of our script:

```
if __name__ == "__main__":
    main("data/inflammation-01.csv")
```

This syntax is a common Python idiom that allows us to run the `main()` function when the script is executed directly from the command line. The `if` statement check if the script is

being run as the main program or if it is being imported as a module in another script. If the script is being run directly, the code inside the `if` block will be executed. If it is being imported, the code inside the `if` block will not be executed.

Now, running `./code/inflammation_stats.py` in command line should output the average, minimum and maximum inflammation per patient for the file `data/inflammation-01.csv`.

> **Note**
>
> Running a Python script in bash is very similar to importing that file in Python. The biggest difference is that we don't expect anything to happen when we import a file, whereas when running a script, we expect to see some output printed to the console.
> In order for a Python script to work as expected when imported or when run as a script, we typically put the part of the script that produces output in the following if statement:
>
> ```
> if __name__ == '__main__':
>  main()  # Or whatever function produces output
> ```
>
> When you import a Python file, `__name__` is the name of that file (e.g., when importing readings.py, `__name__` is `inflammation_stats`). However, when running a script in bash, `__name__` is always set to `__main__` in that script so that you can determine if the file is being imported or run as a script.

15.2 User-defined input

There are some interesting ways to get input from the user:

- `input()` receives input from the keyboard. This means that the input is defined *while* the python script is being executed.
- `sys.argv` takes arguments provided in command line after the name of the program. This means that the input is defined *before* the python script is being executed.
- `argparse` is similar to `sys.argv`, with the advantage of being able to give specific names to arguments.

The `sys.argv` list contains the command-line arguments passed to the script, and `sys.argv[1]` refers to the first argument (the filename in this case).

15.2.1 input

Python stops executing when it comes to the `input()` function, and continues when the user has given some input.

In the `inflammation_stats.py` file , write the following after the end of the `main()` function:

```
filename = input("Enter filename: ")
print("Filename is: " + filename)

if __name__ == "__main__":
    main(filename)
```

Then in the terminal, run:

```
./code/inflammation_stats.py
```

You should be asked, in command line, to enter a filename. When you write it (e.g. `data/inflammation-01.csv`), and press Enter, it should run the `main()` function with the given file.

```
Enter filename: data/inflammation-01.csv
Filename is: data/inflammation-01.csv
min: 0
mean: 6.14875
max: 20
```

15.2.2 sys.argv

To use `sys.argv` you need to import a module called `sys`. It is part of the standard python library, so you should not have to install anything in particular.

Modify the same python script, to add at the beginning of the script:

```
import sys
```

and substitute the end of the script with:

```
print("Filename is: " + sys.argv[1])

if __name__ == "__main__":
    main(sys.argv[1])
```

Then in the terminal, run:

```
./code/inflammation_stats.py data/inflammation-01.csv
```

Arguments are given in command line, separated by `[space]`.

```
Filename is: data/inflammation-01.csv
min: 0
mean: 6.14875
max: 20
```

> **Note**
>
> What is the type of `sys.argv`? Remember that in python index begins at `0`. What do you think is `sys.argv[0]`? Verify!
> Also, what happens if you run `./code/inflammation_stats.py data/inflammation-01.csv data/inflammation-02.csv`?

15.2.3 argparse

Just like for `sys`, you need to import `argparse`.

Modify the same python script, to add at the beginning of the script:

```
import argparse
```

and substitute the end of the script with:

```
parser = argparse.ArgumentParser()
parser.add_argument('--filename', action="store", required=True)
args = parser.parse_args()
print("Filename is: " + args.filename)

if __name__ == "__main__":
    main(args.filename)
```

Then in the terminal, run:

```
./code/inflammation_stats.py --filename data/inflammation-01.csv
```

Arguments are given in command line, but they have specific names.

> **Note**
>
> `argparse` is a very useful module when creating programs! You can easily specify the expected type of argument, whether it is optional or not, and create a help for your script. Check their tutorial for more information.

15.3 Exercise

> **Exercise**
>
> Modify the script to also take `--min`, `--mean` and `--max` as command line arguments. If the argument is given, the corresponding statistic will be calculated and printed. By default, it should not calculate any of the statistics, the user has to specify that in command line.
> If the user runs `./code/inflammation_stats_argparse.py --filename data/inflammation-01.csv --min --max`, it should only calculate and print the minimum and maximum inflammation per patient, but not the mean.
> You can use the `action="store_true"` option in `add_argument()` to create a boolean variable for each statistic.
> For example:
>
> ```
> parser.add_argument('--min', action="store_true")
> ```
>
> This line will create a variable called `args.min`, that will be set to `False` by default. When the user provides the `--min` argument in command line, `args.min` will be set to `True`.
> You should successfully run your script with the following commands:
>
> ```
> ./code/inflammation_stats.py --filename data/inflammation-01.csv --min
> ./code/inflammation_stats.py --filename data/inflammation-01.csv --max
> ./code/inflammation_stats.py --filename data/inflammation-01.csv --min --max
> ./code/inflammation_stats.py --filename data/inflammation-01.csv --min --max --mean
> ```

15.4 Handle multiple filenames

The next step is to teach our program how to handle multiple files.

We want our program to process each file separately, so we need a loop that executes once for each filename. If we specify the files on the command line, the filenames will be in

`args.filename`. Fortunately, `argparse` has a built-in way to handle an unknown number of arguments. We can use the `nargs='+'` option in `add_argument()` to specify that we want to accept one or more filenames as input. This will create a list of filenames in `args.filename`, which we can then loop through.

Here's how it can be done:

```
#!/usr/bin/env python3

import argparse
import pandas as pd

def main(filename, min = True, mean = True, max = True):
    for file in filename:
        print("Processing file:", file)
        data = pd.read_csv(file, index_col=0)
        stats = {}

        if min:
            stats['min'] = data.min(axis=0).min()
        if mean:
            stats['mean'] = data.mean(axis=0).mean()
        if max:
            stats['max'] = data.max(axis=0).max()

        for key, value in stats.items():
            print(f"{key}: {value}")

# Deal with arguments
parser = argparse.ArgumentParser()
parser.add_argument('--filename', nargs='+', action="store", required=True)
parser.add_argument('--min', action="store_true")
parser.add_argument('--max', action="store_true")
parser.add_argument('--mean', action="store_true")
args = parser.parse_args()

if __name__ == "__main__":
    main(args.filename, min=args.min, mean=args.mean, max=args.max)
```

Then in command line, you can run:

```
./code/inflammation_stats.py --filename data/inflammation-01.csv --min --max
./code/inflammation_stats.py --filename data/inflammation-01.csv data/inflammation-02.csv --m
./code/inflammation_stats.py --filename data/inflammation-0*.csv --min --max
```

> **Note**
>
> What is the type of `args.filename` when using `nargs='+'`? What does it contain?

15.5 Testing the code

When coding, a good practice is to test your code on small data files before running it on large datasets. This allows you to check that your code is working as expected and to catch any errors or bugs before they cause problems with larger datasets.

Indeed, your code might not have any syntax errors, but it might not be doing what you think it is doing. For example, you might think that your code is calculating the mean of a dataset, but in reality, it is calculating the sum. If you run your code on a large dataset, you might not notice this error until it is too late.

Here we can test our code on a smaller file that has the same structure as the larger files, but only contains a few lines of data. This way, we can easily check that our code is calculating the mean correctly for each line, and that it is doing what we expect it to do.

The file `data/small-01.csv` contains the following data:

```
Day1,Day2,Day3
Patient1,0,0,1
Patient2,0,1,2
```

We can run:

```
./code/inflammation_stats.py --filename data/small-01.csv --min --max --mean
```

There are two more files `small-02.csv` and `small-03.csv` that you can use to test your code.

It is also important to test your code on edge cases, which are cases that are at the extreme ends of the input space. For example, if your code is supposed to handle missing values, you should test it on a file that contains missing values to make sure that it is handling them correctly.

Even if your documentation is very clear, the user might not behave as you expect them to, and might provide input that is not in the format you expected. For example, if your code is supposed to take a filename as input, the user might provide a filename that does not exist, or a filename that is in a different format than what you expected. It is important to test your code on such edge cases to make sure that it is handling them correctly and providing useful error messages to the user.

16 Errors and Exceptions

Every programmer encounters errors, both those who are just beginning, and those who have been programming for years. Encountering errors and exceptions can be very frustrating at times, and can make coding feel like a hopeless endeavour. However, understanding what the different types of errors are and when you are likely to encounter them can help a lot. Once you know why you get certain types of errors, they become much easier to fix.

16.1 Tracebacks

Errors in Python have a very specific form, called a traceback. Let's examine one by providing a filename that does not exist:

```
./code/inflammation_stats.py --filename data/small-06.csv --min --max --mean
```

```
Processing file: data/small-06.csv
Traceback (most recent call last):
  File "/Volumes/projects/PCa_SingleCell/utils/workshop/python-intro/./code/inflammation_sta
    main(args.filename, min=args.min, mean=args.mean, max=args.max)
  File "/Volumes/projects/PCa_SingleCell/utils/workshop/python-intro/./code/inflammation_sta
    data = pd.read_csv(file, index_col=0)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/gilbartv/miniconda3/lib/python3.11/site-packages/pandas/io/parsers/readers.py
    return _read(filepath_or_buffer, kwds)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/gilbartv/miniconda3/lib/python3.11/site-packages/pandas/io/parsers/readers.py
    parser = TextFileReader(filepath_or_buffer, **kwds)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/gilbartv/miniconda3/lib/python3.11/site-packages/pandas/io/parsers/readers.py
    self._engine = self._make_engine(f, self.engine)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/gilbartv/miniconda3/lib/python3.11/site-packages/pandas/io/parsers/readers.py
    self.handles = get_handle(
                   ^^^^^^^^^^^
  File "/Users/gilbartv/miniconda3/lib/python3.11/site-packages/pandas/io/common.py", line 87
```

```
    handle = open(
             ^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'data/small-06.csv'
```

Python tells us the category or type of error (in this case, it is an `FileNotFoundError`) and a more detailed error message (in this case, it says `[Errno 2] No such file or directory: 'data/small-06.csv'`).

This particular traceback has many levels. The traceback also shows us the exact line of code where the error occurred, and the sequence of function calls that led to that line of code being executed.

The lines in our code that caused the error are `main(args.filename, min=args.min, mean=args.mean, max=args.max)`, and then `pd.read_csv(file, index_col=0)`. Indeed `main()` is the function that we called in command line, and inside `main()`, the error occurred when trying to read the file with `pd.read_csv()`. It gives more or the subsequent calls that were made inside the `pandas` library to get to the line of code that caused the error, but we can ignore those for now.

> **Note**
>
> The length of the error message does not reflect severity, rather, it indicates that your program called many subsequent functions before it encountered the error.

One reason for receiving this `FileNotFoundError` error is that you specified an incorrect path to the file. For example, if I am currently in a folder called `myproject`, and I have a file in `myproject/writing/myfile.txt`, but I try to open `myfile.txt`, this will fail. The correct path would be `writing/myfile.txt`.

It is also possible that the file name or its path contains a typo.

If you encounter an error and don't know what it means, it is still important to read the traceback closely. That way, if you fix the error, but encounter a new one, you can tell that the error changed. Additionally, sometimes knowing where the error occurred is enough to fix it, even if you don't entirely understand the message.

16.2 Type of Exceptions

If you do encounter an error you don't recognize, try looking at the official documentation on errors also called `Exceptions` in Python. However, note that you may not always be able to find the error there, as it is possible to create custom errors. In that case, hopefully the custom error message is informative enough to help you figure out what went wrong.

Here is a table of some of the built-in exceptions in python.

Exception	Description
`IndexError`	Raised when the index of a sequence is out of range.
`KeyError`	Raised when a key is not found in a dictionary.
`KeyboardInterrupt`	Raised when the user hits the interrupt key (Ctrl+c or Delete).
`NameError`	Raised when a variable is not found in the local or global scope.
`TypeError`	Raised when a function or operation is applied to an object of an incorrect type.
`StatementError`	Raised when statements are in an order or contain characters not expected by the programming language.
`ValueError`	Raised when a function receives an argument of the correct type but of an incorrect value.
`RuntimeError`	Raised when an error occurs that do not belong to any specific exceptions.
`Exception`	Base class of exceptions.

16.3 Exercise

! Exercise

Try the following code and understand what the error is:

```python
# Code A
def print_message(day):
    messages = [
        'Hello, world!',
        'Today is Tuesday!',
        'It is the middle of the week.',
        'Today is Donnerstag in German!',
        'Last day of the week!',
        'Hooray for the weekend!',
        'Aw, the weekend is almost over.'
    ]
    print(messages[day])

def print_sunday_message():
    print_message(7)

print_sunday_message()
```

```
IndexError: list index out of range
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[57], line 17
     14 def print_sunday_message():
     15     print_message(7)
---> 17 print_sunday_message()
Cell In[57], line 15, in print_sunday_message()
     14 def print_sunday_message():
---> 15     print_message(7)
Cell In[57], line 12, in print_message(day)
      2 def print_message(day):
      3     messages = [
      4         'Hello, world!',
      5         'Today is Tuesday!',
   (...)
     10         'Aw, the weekend is almost over.'
     11     ]
---> 12     print(messages[day])
IndexError: list index out of range
```

```python
# Code B
def some_function()
    msg = 'hello, world!'
    print(msg)
     return msg
```

```
SyntaxError: expected ':' (174094176.py, line 2)
  Cell In[58], line 2
    def some_function()
                       ^
SyntaxError: expected ':'
```

```python
# Code C
def some_function():
    msg = 'hello, world!'
    print(msg)
        return msg
```

```
TabError: inconsistent use of tabs and spaces in indentation (1281988140.py, line 5)
  Cell In[59], line 5
    return msg
    ^
TabError: inconsistent use of tabs and spaces in indentation
```

```python
# Code D
print(a)
```

```
NameError: name 'a' is not defined
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[60], line 2
      1 # Code D
----> 2 print(a)
NameError: name 'a' is not defined
```

```python
# Code E
for number in range(10):
    total = total + number
print('The count is:', count)
```

```
NameError: name 'total' is not defined
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[61], line 3
      1 # Code E
      2 for number in range(10):
----> 3     total = total + number
      4 print('The count is:', count)
NameError: name 'total' is not defined
```

```
# Code F
Count = 0
for number in range(10):
    count = count + number
print('The count is:', count)
```

```
NameError: name 'count' is not defined
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[62], line 4
      2 Count = 0
      3 for number in range(10):
----> 4     count = count + number
      5 print('The count is:', count)
NameError: name 'count' is not defined
```

17 Defensive Programming

17.1 Exceptions Handling

It is possible to handle errors (in python, they are also called exceptions), using the following statements:

- `try` to test a block of code for errors
- `except` to handle the error
- `else` to excute code if there is no error
- `finally` to excute code, regardless of the result of the try and except blocks

```
# The try block will generate an exception, because some_undefined_variable is not defined:
try:
  print(some_undefined_variable)
except:
  print("Oops... Something went wrong")
```

```
Oops... Something went wrong
```

```
# Without the try block, the program will crash and raise an error:
print(some_undefined_variable)
```

```
NameError: name 'some_undefined_variable' is not defined
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[64], line 2
      1 # Without the try block, the program will crash and raise an error:
----> 2 print(some_undefined_variable)
NameError: name 'some_undefined_variable' is not defined
```

```
try:
  print(some_undefined_variable)
except:
  print("Oops... Something went wrong")
```

```
else:
  print("Nothing went wrong")
finally:
  print("The 'try except' is finished")
```

```
Oops... Something went wrong
The 'try except' is finished
```

You can use the `Exceptions` types to be more specific about the type of exception occurring.

```
try:
  print(some_undefined_variable)
except NameError:
  print("A variable is not defined")
except:
  print("Oops... Something went wrong")
else:
  print("Nothing went wrong")
finally:
  print("The 'try except' is finished")
```

```
A variable is not defined
The 'try except' is finished
```

You can also use the `Exceptions` types to throw an exception if a condition occurs, by using the `raise` keyword.

```
x = "hello"
try:
  if not isinstance(x, int):
    raise TypeError("Only integers are allowed")
  if x < 0:
    raise ValueError("Sorry, no numbers below zero")
  print(x, "is a positive integer.")
except NameError:
  print("A variable is not defined")
else:
  print("Nothing went wrong")
finally:
  print("The 'try except' is finished")
```

```
The 'try except' is finished

TypeError: Only integers are allowed
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[67], line 4
      2 try:
      3   if not isinstance(x, int):
----> 4     raise TypeError("Only integers are allowed")
      5   if x < 0:
      6     raise ValueError("Sorry, no numbers below zero")
TypeError: Only integers are allowed
```

17.2 Exercise

! Exercise

Let's make our previous function even better by adding some exception handling. Raise a `TypeError` if the input `filename` is not a string. To verify that a variable is of a certain type, you can use the `isinstance()` function. For example, `isinstance(x, str)` will return `True` if `x` is a string, and `False` otherwise.
With the input given below, the output and errors should be:

```python
import pandas as pd

def main(filename, min = True, mean = True, max = True):
    print("Processing file:", filename)
    data = pd.read_csv(filename, index_col=0)
    stats = {}
    if min:
        stats['min'] = data.min(axis=0).min()
    if mean:
        stats['mean'] = data.mean(axis=0).mean()
    if max:
        stats['max'] = data.max(axis=0).max()
    for key, value in stats.items():
        print(f"{key}: {value}")

main(filename = 5474)
main(filename = "data/inflammation-01.csv")
```

```
TypeError: Filename must be a string, 5474 is of type <class 'int'>
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[68], line 18
     15     for key, value in stats.items():
     16         print(f"{key}: {value}")
---> 18 main(filename = 5474)
     19 main(filename = "data/inflammation-01.csv")
Cell In[68], line 5, in main(filename, min, mean, max)
      3 def main(filename, min = True, mean = True, max = True):
      4     if not isinstance(filename, str):
----> 5         raise TypeError(f"Filename must be a string, {filename} is of type {type(fi
      6     print("Processing file:", filename)
      7     data = pd.read_csv(filename, index_col=0)
TypeError: Filename must be a string, 5474 is of type <class 'int'>
```

18 Debugging

18.1 Know what the code is supposed to do

The first step in debugging something is to know what it's supposed to do. "My program doesn't work" isn't good enough: in order to diagnose and fix problems, we need to be able to tell correct output from incorrect. If we can write a test case for the failing case — i.e., if we can assert that with these inputs, the function should produce that result — then we're ready to start debugging. If we can't, then we need to figure out how we're going to know when we've fixed things.

But writing test cases for scientific software is frequently harder than writing test cases for commercial applications, because if we knew what the output of the scientific code was supposed to be, we wouldn't be running the software: we'd be writing up our results and moving on to the next program. In practice, scientists tend to do the following:

- Test with simplified data. Before doing statistics on a real data set, we should try calculating statistics for a single record, for two identical records, for two records whose values are one step apart, or for some other case where we can calculate the right answer by hand.
- Test a simplified case. If our program is supposed to simulate magnetic eddies in rapidly-rotating blobs of supercooled helium, our first test should be a blob of helium that isn't rotating, and isn't being subjected to any external electromagnetic fields. Similarly, if we're looking at the effects of climate change on speciation, our first test should hold temperature, precipitation, and other factors constant.
- Compare to an oracle. A test oracle is something whose results are trusted, such as experimental data, an older program, or a human expert. We use test oracles to determine if our new program produces the correct results. If we have a test oracle, we should store its output for particular cases so that we can compare it with our new results as often as we like without re-running that program.
- Check conservation laws. Mass, energy, and other quantities are conserved in physical systems, so they should be in programs as well. Similarly, if we are analyzing patient data, the number of records should either stay the same or decrease as we move from one analysis to the next (since we might throw away outliers or records with missing values). If "new" patients start appearing out of nowhere as we move through our pipeline, it's probably a sign that something is wrong.

- Visualize. Data analysts frequently use simple visualizations to check both the science they're doing and the correctness of their code. This should only be used for debugging as a last resort, though, since it's very hard to compare two visualizations automatically.

18.2 Make it Fail Everytime

We can only debug something when it fails, so the second step is always to find a test case that makes it fail every time. The "every time" part is important because few things are more frustrating than debugging an intermittent problem: if we have to call a function a dozen times to get a single failure, the odds are good that we'll scroll past the failure when it actually occurs.

As part of this, it's always important to check that our code is "plugged in", i.e., that we're actually exercising the problem that we think we are. Every programmer has spent hours chasing a bug, only to realize that they were actually calling their code on the wrong data set or with the wrong configuration parameters, or are using the wrong version of the software entirely. Mistakes like these are particularly likely to happen when we're tired, frustrated, and up against a deadline, which is one of the reasons late-night (or overnight) coding sessions are almost never worthwhile.

18.3 Make it Fail Fast

If it takes 20 minutes for the bug to surface, we can only do three experiments an hour. This means that we'll get less data in more time and that we're more likely to be distracted by other things as we wait for our program to fail, which means the time we are spending on the problem is less focused. It's therefore critical to make it fail fast.

As well as making the program fail fast in time, we want to make it fail fast in space, i.e., we want to localize the failure to the smallest possible region of code:

- The smaller the gap between cause and effect, the easier the connection is to find. Many programmers therefore use a divide and conquer strategy to find bugs, i.e., if the output of a function is wrong, they check whether things are OK in the middle, then concentrate on either the first or second half, and so on.
- N things can interact in N! different ways, so every line of code that isn't run as part of a test means more than one thing we don't need to worry about.

18.4 Change One Thing at a Time

Replacing random chunks of code is unlikely to do much good. (After all, if you got it wrong the first time, you'll probably get it wrong the second and third as well.) Good programmers therefore change one thing at a time, for a reason. They are either trying to gather more information ("is the bug still there if we change the order of the loops?") or test a fix ("can we make the bug go away by sorting our data before processing it?").

Every time we make a change, however small, we should re-run our tests immediately, because the more things we change at once, the harder it is to know what's responsible for what (those N! interactions again). And we should re-run all of our tests: more than half of fixes made to code introduce (or re-introduce) bugs, so re-running all of our tests tells us whether we have regressed.

18.5 Keep Track of What You've Done

Good scientists keep track of what they've done so that they can reproduce their work, and so that they don't waste time repeating the same experiments or running ones whose results won't be interesting. Similarly, debugging works best when we keep track of what we've done and how well it worked. If we find ourselves asking, "Did left followed by right with an odd number of lines cause the crash? Or was it right followed by left? Or was I using an even number of lines?" then it's time to step away from the computer, take a deep breath, and start working more systematically.

Records are particularly useful when the time comes to ask for help. People are more likely to listen to us when we can explain clearly what we did, and we're better able to give them the information they need to be useful.

18.6 Ask for help

And speaking of help: if we can't find a bug in 10 minutes, we should be humble and ask for help. Explaining the problem to someone else is often useful, since hearing what we're thinking helps us spot inconsistencies and hidden assumptions. If you don't have someone nearby to share your problem description with, get a rubber duck!

Asking for help also helps alleviate confirmation bias. If we have just spent an hour writing a complicated program, we want it to work, so we're likely to keep telling ourselves why it should, rather than searching for the reason it doesn't. People who aren't emotionally invested in the code can be more objective, which is why they're often able to spot the simple mistakes we have overlooked.

Part of being humble is learning from our mistakes. Programmers tend to get the same things wrong over and over: either they don't understand the language and libraries they're working with, or their model of how things work is wrong. In either case, taking note of why the error occurred and checking for it next time quickly turns into not making the mistake at all.

If we train ourselves to avoid making some kinds of mistakes, to break our code into modular, testable chunks, and to turn every assumption (or mistake) into an assertion, it will actually take us less time to produce working programs, not more.

19 Final tips and resources

Here are a couple of tips:

- Leave comments (think of your future self)
- Be consistent (quotes, indents…)
- Break down one complex task into lots of (easy) small tasks
- When using functions you are not comfortable with, verify the output and make sure it does what you expect in with small examples
- Don't re-invent the wheel, for common tasks, it's likely that a function already exists
- Read the documentation when using a new package or function
- Google It! Use the correct programming vocabulary to increase your chances of finding an answer. If you don't find anything, try wording it differently.
- Prompt it to AI! It works generally well to explain a code, and for small tasks using famous packages.
- The easiest way to learn is by example, so follow a tutorial with the example data, and then try to apply it to your own

You can follow some free tutorials on:

- Code Academy
- EdX
- Youtube!

References

Here are some references and ressources that greatly inspired this class :

- Python doc
- w3schools
- pythonforbiologists
- justinbois's Bootcamp
- Software carpentry 1
- Software carpentry 2